4 – Métodos de Pesquisa

Carlos Ramos

csr@isep.ipp.pt

Unidade Curricular: Inteligência Artificial

Aulas Teóricas

Curso: Licenciatura em Engenharia Informática (ISEP)



Métodos Informados de Pesquisa

Neste capítulo vamos abordar os métodos informados de pesquisa sistemática

Iremos considerar 3 métodos:

- Primeiro o Melhor
- Branch & Bound
- A*

Iremos ainda apresentar um método de combinação de soluções, classificado dentro dos métodos de Computação Evolucionária, que corresponde ao método dos Algoritmos Genéticos

Primeiro o Melhor

- Vamos descrever agora o primeiro método dito “informado” de pesquisa. O método “Primeiro o Melhor” assemelha-se ao Primeiro em Profundidade, a única diferença reside no facto da decisão sobre qual ramo seguir ser feita com base num critério de decisão local

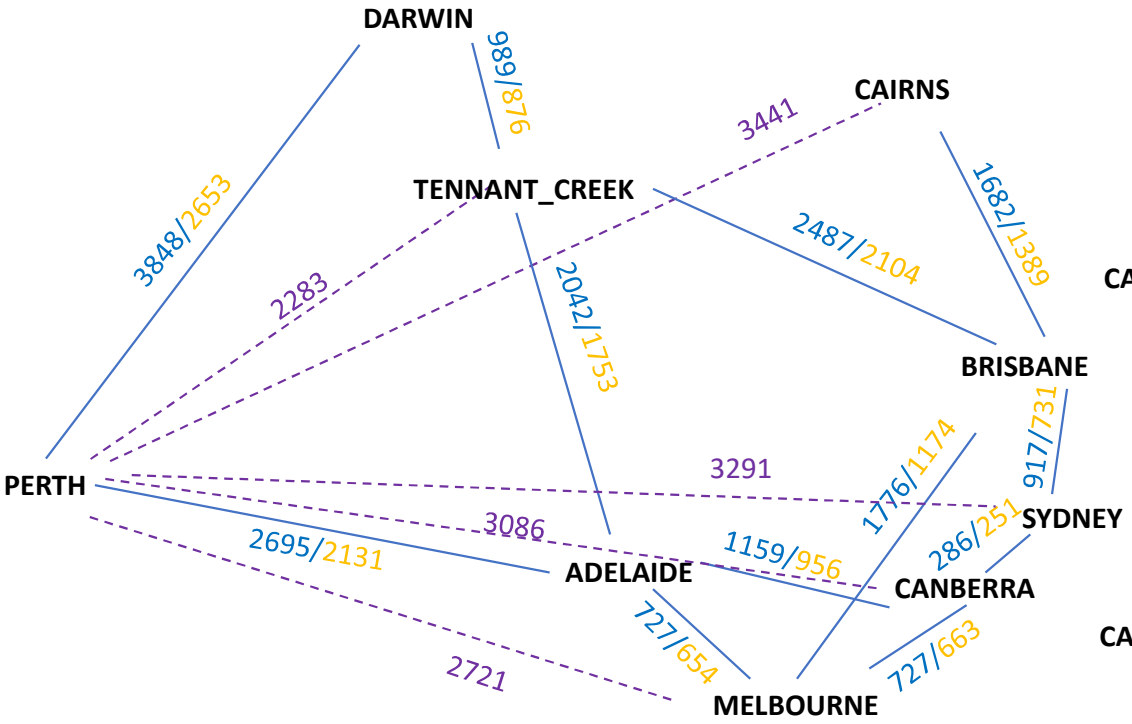
- A ideia que está por trás é a de em caso de indecisão sobre qual caminho vamos seguir deve-se optar por aquele que pareça mais promissor. Trata-se de uma decisão feita localmente com base numa informação particular

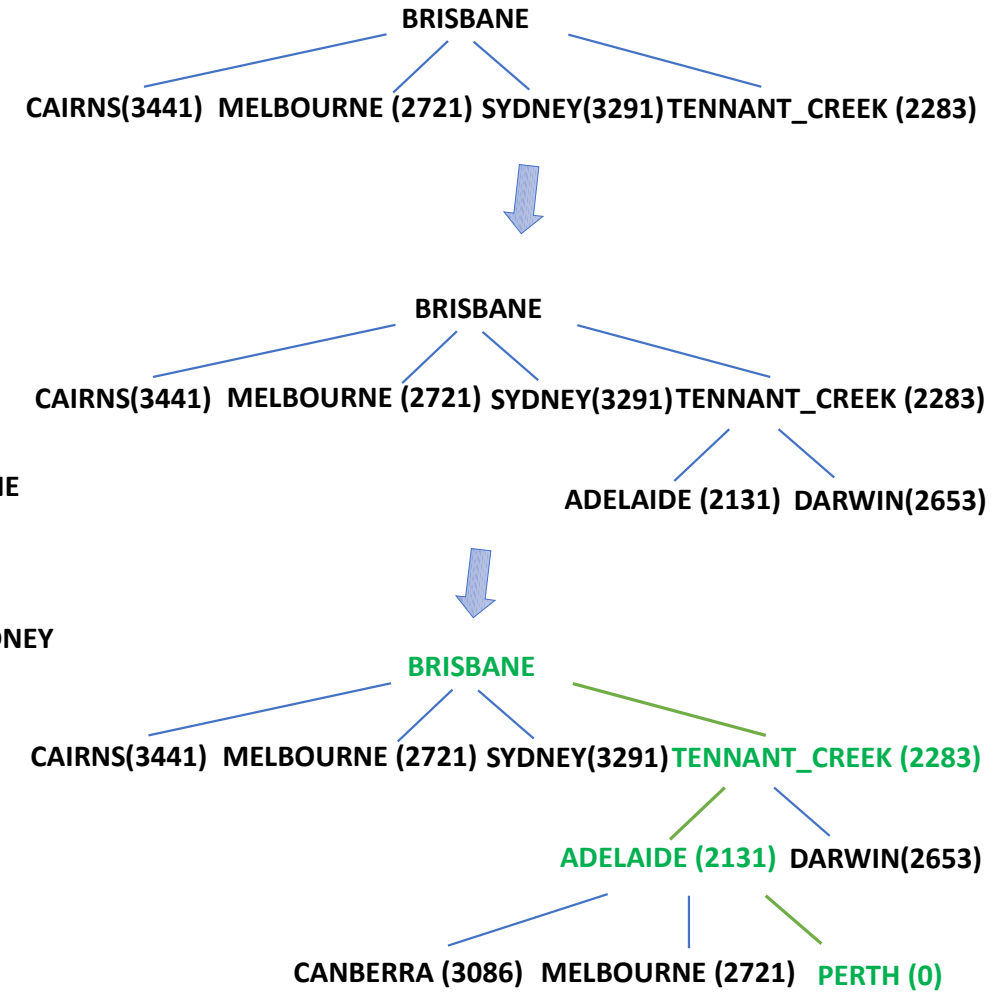
- É então necessário dispor de um valor numérico que avalie o custo ou o ganho que se obtém pelo facto de seguir por um dado caminho

- Dada a possibilidade de escolher caminhos que não levem ao estado final pretendido é sempre aconselhável implementar o método com retrocesso, ou seja, se chegar a um “beco sem saída” volta atrás e explora outra alternativa

Primeiro o Melhor com Retrocesso

Objetivo: ir de BRISBANE para PERTH





Vamos assumir que vamos para o nó que se encontra mais próximo do destino pretendido

Para este exemplo vamos precisar das distâncias a PERTH

Encontra solução ? Encontra melhor solução ?

Best First Search (BestFS) Implementação Prolog

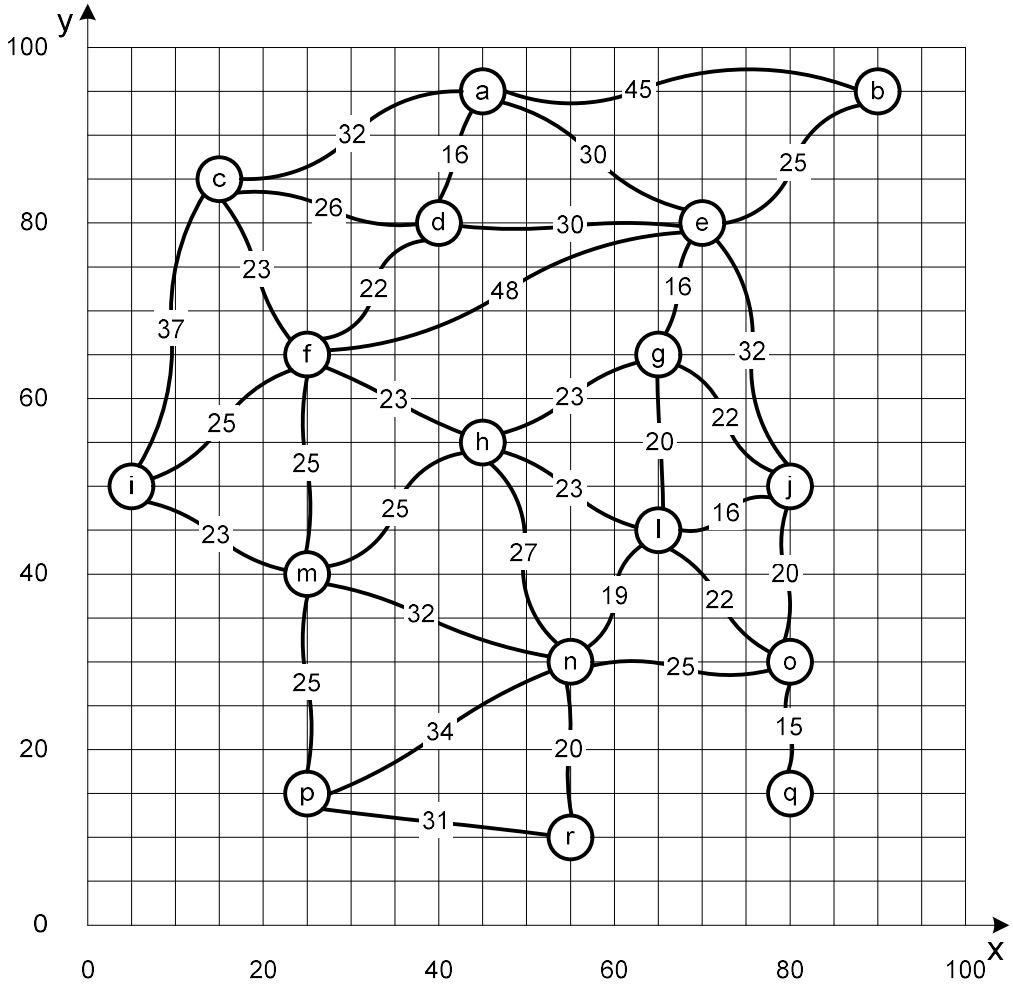
```
bestfs(Orig,Dest,Cam):-
        bestfs2(Dest,[Orig],Cam).

%condicao final: destino = nó à cabeça do caminho actual
bestfs2(Dest,[Dest|T],Cam):- !,
        %caminho actual está invertido
        reverse([Dest|T],Cam).

bestfs2(Dest,LA,Cam):-
        LA=[Act|_],
        %calcular todos os nodos adjacentes nao visitados e
        % guardar um tuplo com estimativa e novo caminho
        findall((EstX,[X|LA]),
                ((edge(Act,X);edge(X,Act)),
                  \+ member(X,LA), estimativa(X,Dest,EstX)),
                Novos),
        %ordenar pela estimativa
        sort(Novos,NovosOrd),
        %extrair o melhor que está à cabeça
        NovosOrd = [(_,Melhor)|_],
        %chamada recursiva
        bestfs2(Dest,Melhor,Cam).
```

Best First Search (BestFS) (Outro Exemplo)

```
%node(Nodo,PosX,PosY)
node(a,45,95).
node(b,90,95).
...
%edge(Nodo1,Nodo2,Custo)
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
...
```



Best First Search (BestFS) – versão c/custos
Implementação Prolog / Soluções

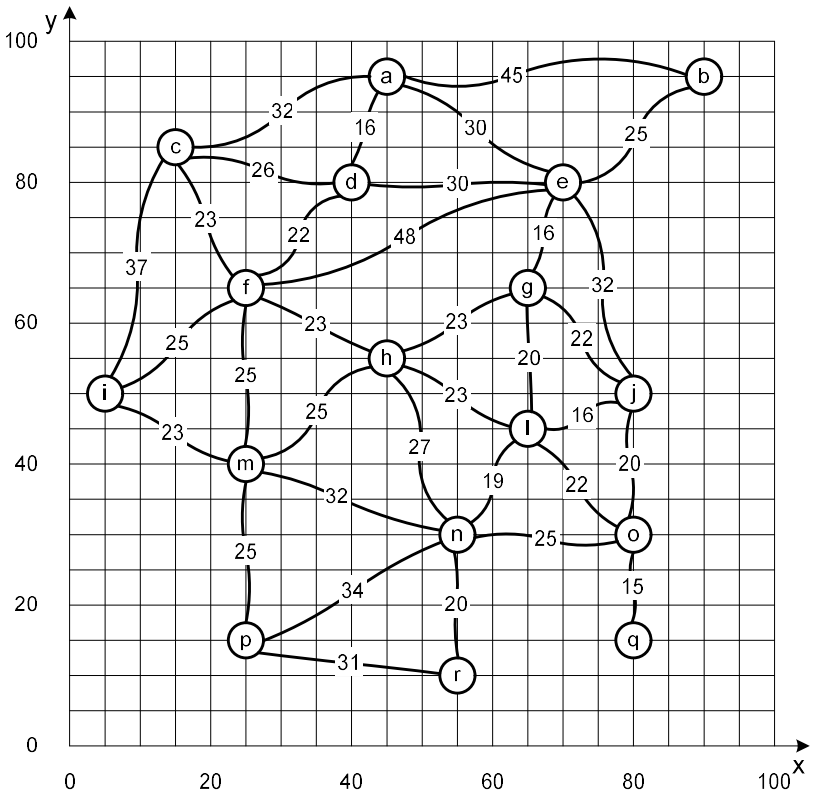
```
?- bestfs(a,r,Cam,Custo).
Cam = [a, d, f, m, n, r],
Custo = 115.
```

```
bestfs(Orig,Dest,Cam,Custo):-
        bestfs2(Dest,(0,[Orig]),Cam,Custo).

bestfs2(Dest,(Custo,[Dest|T]),Cam,Custo):-
        !,
        reverse([Dest|T],Cam).

bestfs2(Dest,(Ca,LA),Cam,Custo):-
        LA=[Act|_],
        findall((EstX,CaX,[X|LA]),
        ((edge(Act,X,CX); edge(X,Act,CX)),
        \+member(X,LA),estimativa(X,Dest,EstX),
        CaX is Ca+CX),Novos),

        sort(Novos,NovosOrd),
        NovosOrd = [(_,CM,Melhor)|_],
        bestfs2(Dest,(CM,Melhor),Cam,Custo).
```



Best First Search (BestFS)
Estimativa

```
estimativa(Nodo1,Nodo2,Estimativa):-
        node(Nodo1,X1,Y1),
        node(Nodo2,X2,Y2),
        Estimativa is sqrt((X1-X2)^2+(Y1-Y2)^2).
```

Best First Search (BestFS) – versão c/custos
Implementação Prolog / Soluções

```
?- bestfs(a,r,Cam,Custo).
Cam = [a, d, f, m, n, r],
Custo = 115.
```

```
bestfs(Orig,Dest,Cam,Custo):-
        bestfs2(Dest,(0,[Orig]),Cam,Custo).

bestfs2(Dest,(Custo,[Dest|T]),Cam,Custo):-
        !,
        reverse([Dest|T],Cam).

bestfs2(Dest,(Ca,LA),Cam,Custo):-
        LA=[Act|_],
        findall((EstX,CaX,[X|LA]),
        ((edge(Act,X,CX); edge(X,Act,CX)),
          \+member(X,LA),estimativa(X,Dest,EstX),
        CaX is Ca+CX),Novos),

        sort(Novos,NovosOrd),
        NovosOrd = [(_,CM,Melhor)|_],
        bestfs2(Dest,(CM,Melhor),Cam,Custo).
```

Esta versão só encontra o caminho se as opções forem para uma alternativa que tenha conectividade com a solução

Experimente cortar as ligações de m para n, n para p e p para r

Dito de outro modo se ocorrer uma falha não há retrocesso para tentar outra alternativa
Isso porque LA=[Act|_] só dá a possibilidade de escolher o elemento que está à cabeça na lista

Mas a solução é simples basta substituir LA=[Act|_] por member(Act,LA)

Branch & Bound

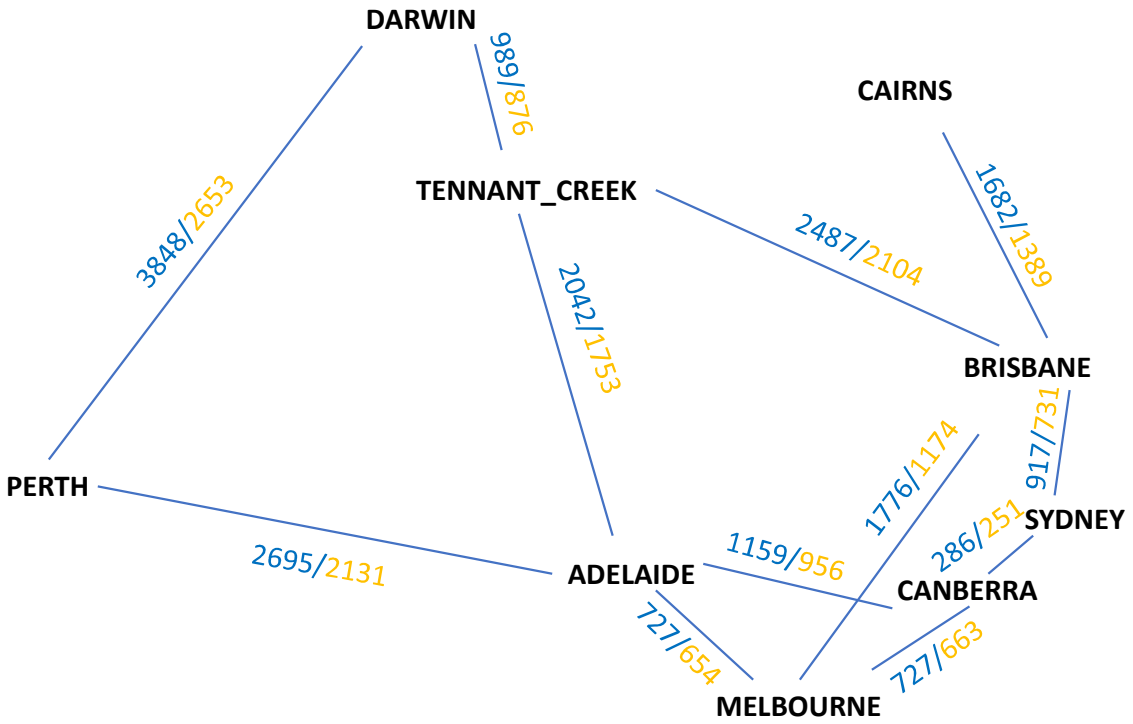
- O Branch and Bound é um método de pesquisa que corresponde ao método "Primeiro o Melhor" com avaliação de transições locais (explorar o próximo nó que está mais próximo do ponto de partida – raiz) mas com possibilidade de alterar a qualquer momento o nó sobre qual se vai considerar a próxima expansão

- Quer isso dizer que o próximo nó que se expande não é obrigatoriamente um descendente do último nó expandido

- A comparação entre os nós candidatos à expansão é feita com base no valor acumulado do custo ou do ganho desde a raiz até esses nós

Vamos usar novamente a rede de Auto-Estradas da Austrália

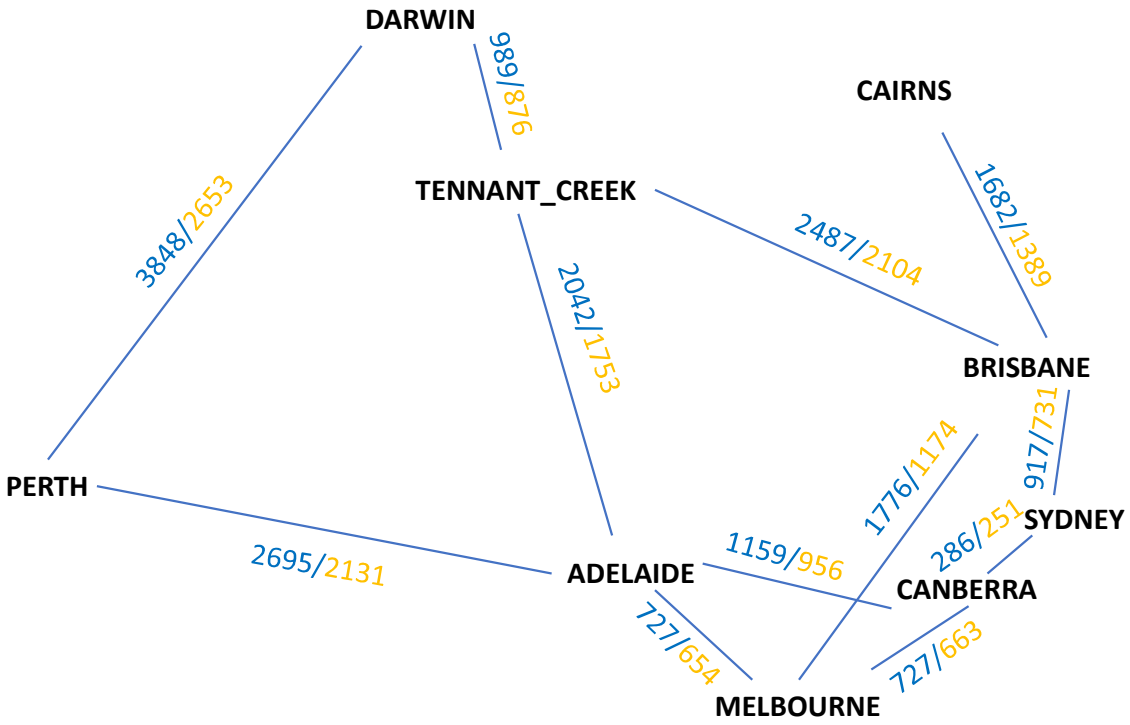


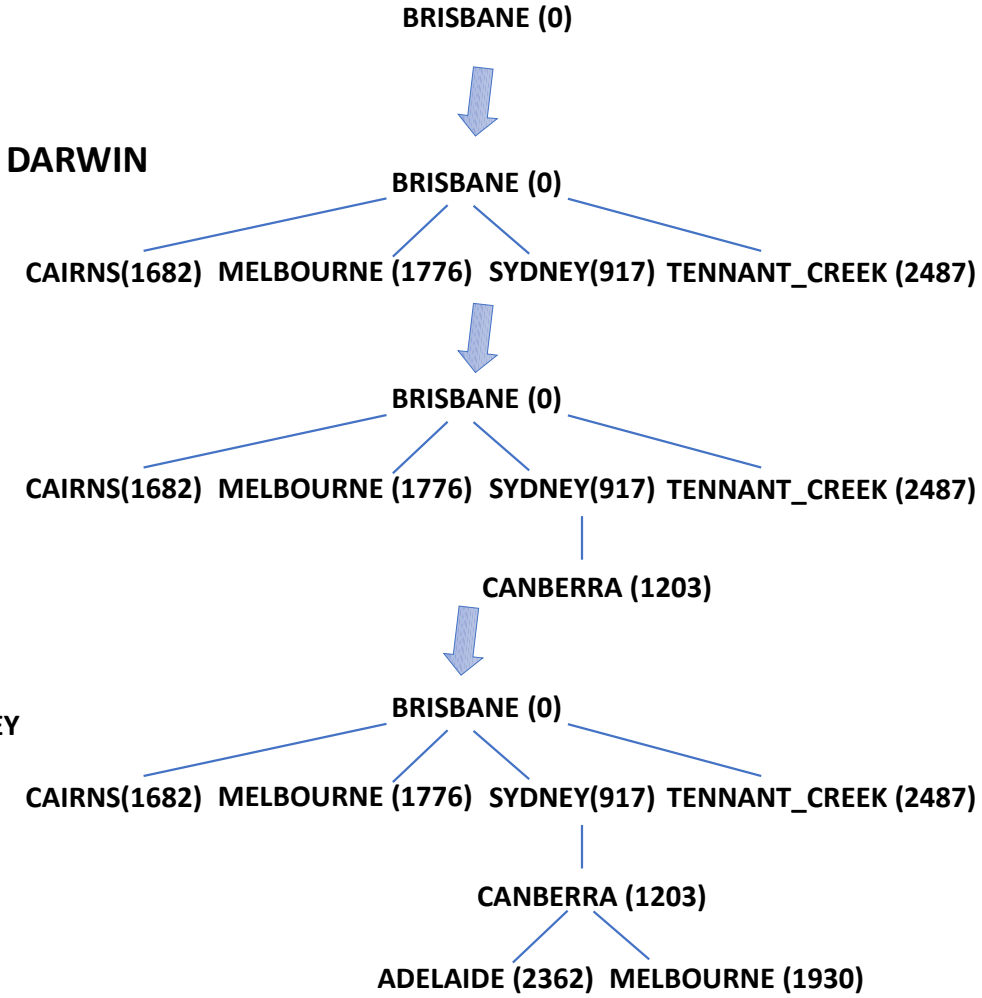


Distância por estrada/Distância menor

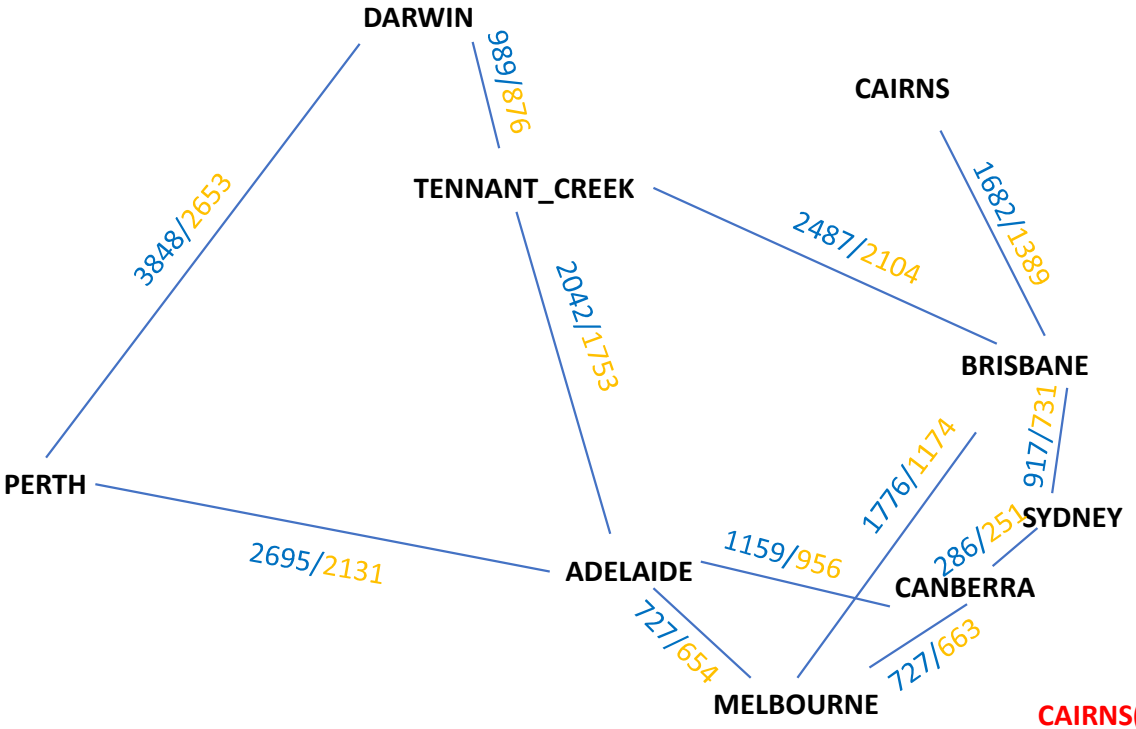
Branch&Bound

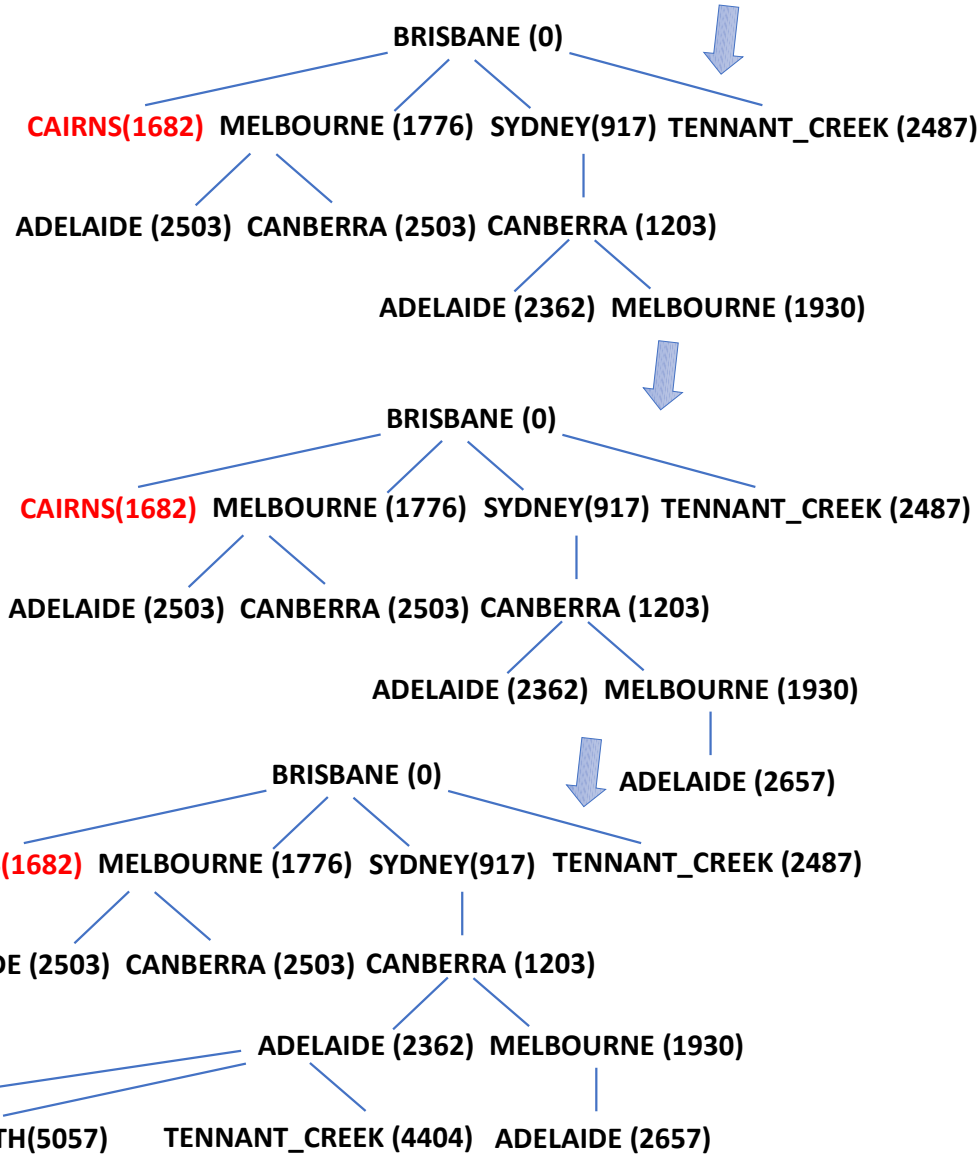
Objetivo: ir de BRISBANE para DARWIN



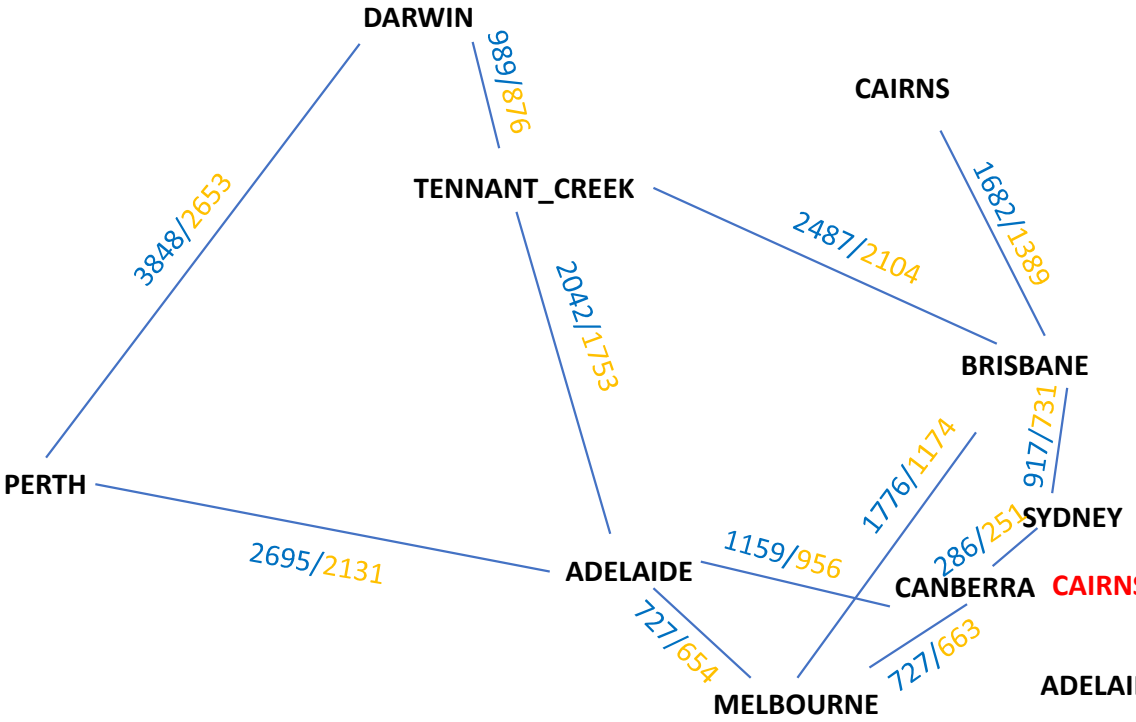


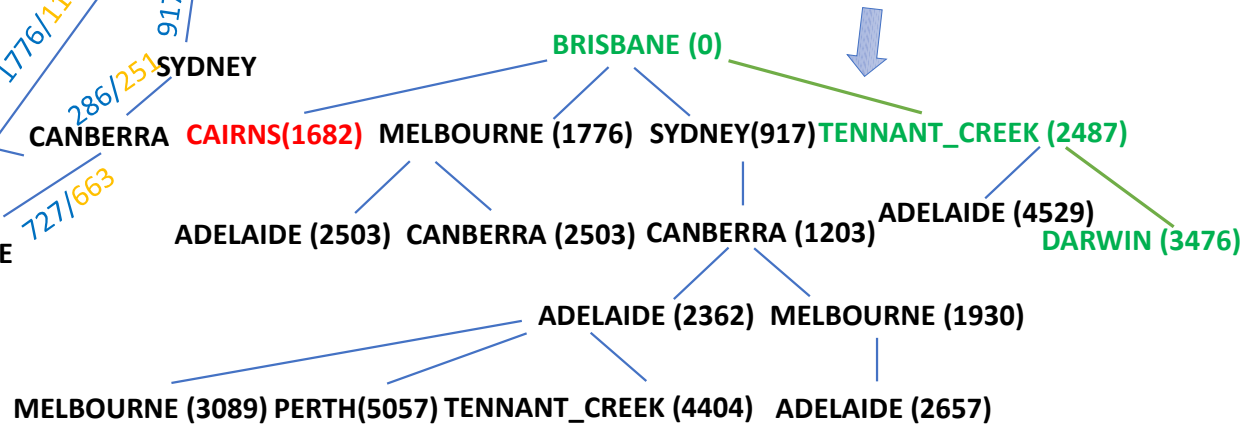
Branch&Bound





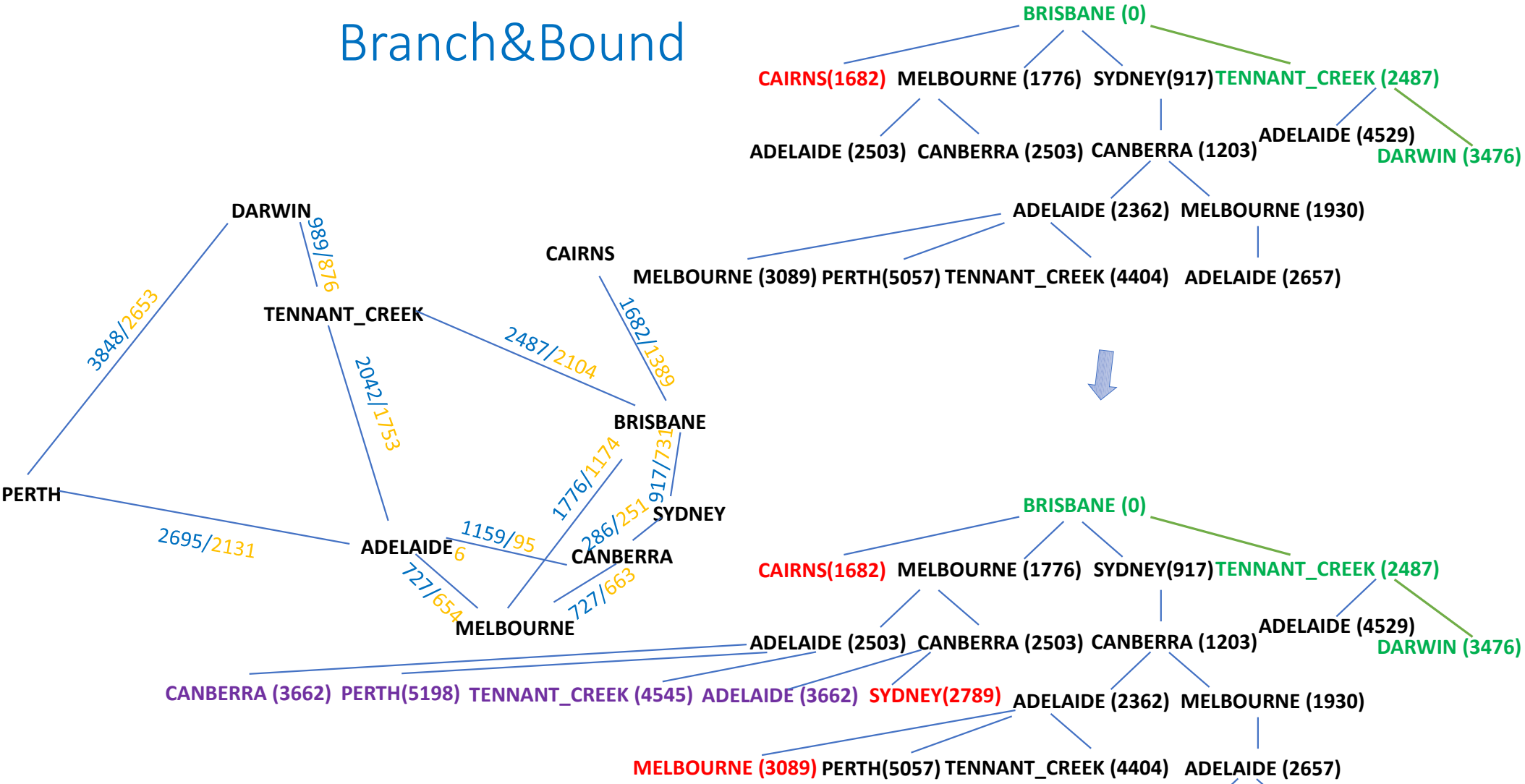
Branch&Bound





Se houver solução o método encontra solução ?
Encontra a melhor solução ?
O que acontece quando temos uma dimensão muito elevada?
Quando podemos garantir que a melhor solução foi atingida ?

Branch&Bound



Só garantimos que temos a melhor solução quando em todos os nós folha
tivermos um custo acumulado igual ou superior ao da melhor solução até ao momento

Branch and Bound
Implementação Prolog

```
bnb(Orig,Dest,Cam,Custo):- bnb2(Dest,[(0,[Orig])],Cam,Custo).

%condicao final: destino = nó à cabeça do caminho actual
bnb2(Dest,[(Custo,[Dest|T])|_],Cam,Custo):-
        %caminho actual está invertido
        reverse([Dest|T],Cam).

bnb2(Dest,[(Ca,LA)|Outros],Cam,Custo):-
        LA=[Act|_],
        %calcular todos os nodos adjacentes nao visitados e
        %gerar tuplos c/ um caminho novo juntado o nodo + caminho actual

        % o custo de cada caminho é o custo acumulado + peso do ramo
        findall((CaX,[X|LA]),
        (Dest\==Act, (edge(Act,X,CustoX); edge(X,Act,CustoX)),
         \+ member(X,LA),CaX is CustoX + Ca),Novos),
        %os novos caminhos sao adicionados aos caminhos não explorados
        append(Outros,Novos,Todos),
        %a ordenação (não sendo o mais eficiente) garante que
        % o melhor caminho fica na primeira posição
        sort(Todos,TodosOrd),
        %chamada recursiva
        bnb2(Dest,TodosOrd,Cam,Custo).
```

Branch and Bound
Implementação Prolog

```
bnb(Orig,Dest,Cam,Custo):-
      bnb2(Dest,[(0,[Orig])],Cam,Custo).

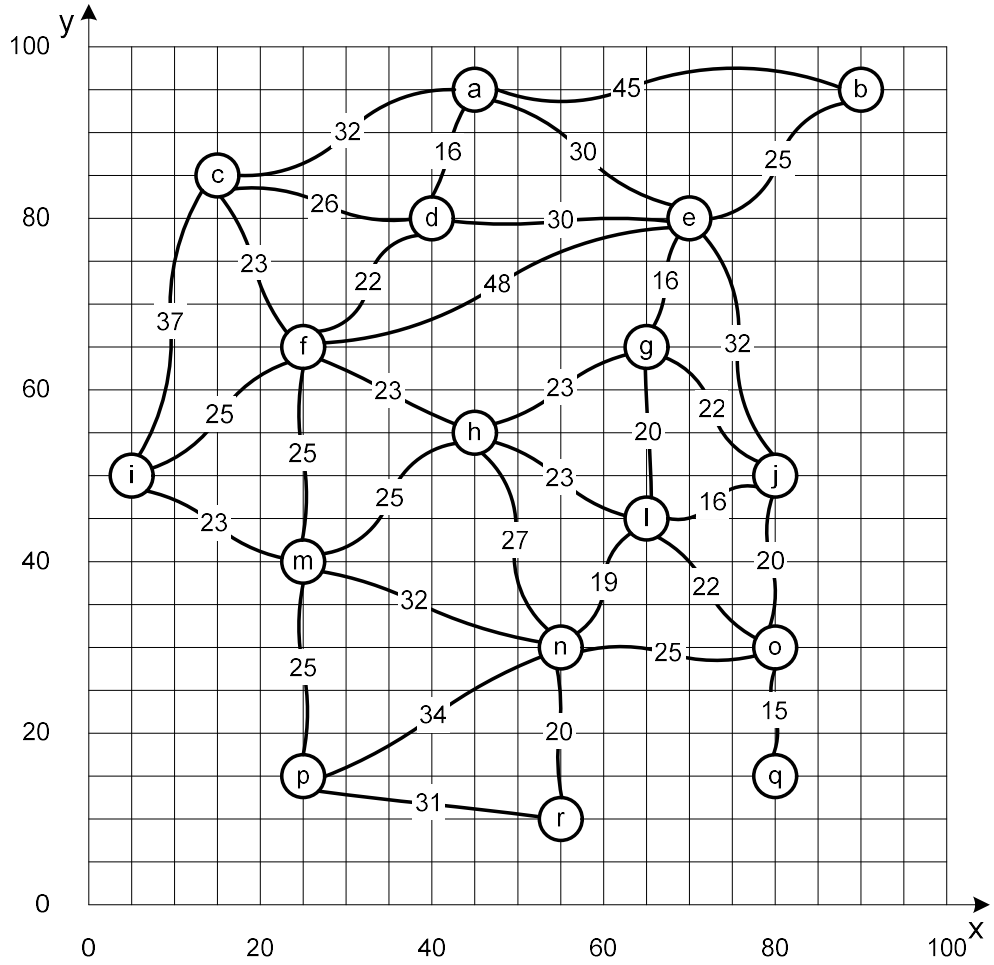
bnb2(Dest,[(Custo,[Dest|T])|_],Cam,Custo):-
      reverse([Dest|T],Cam).

bnb2(Dest,[(Ca,LA)|Outros],Cam,Custo):-
      LA=[Act|_],
      findall((CaX,[X|LA]),
      (Dest\==Act, (edge(Act,X,CustoX);
      edge(X,Act,CustoX)),
       \+ member(X,LA),
      CaX is CustoX + Ca),Novos),
      append(Outros,Novos,Todos),
      sort(Todos,TodosOrd),
      bnb2(Dest,TodosOrd,Cam,Custo).
```

Branch and Bound

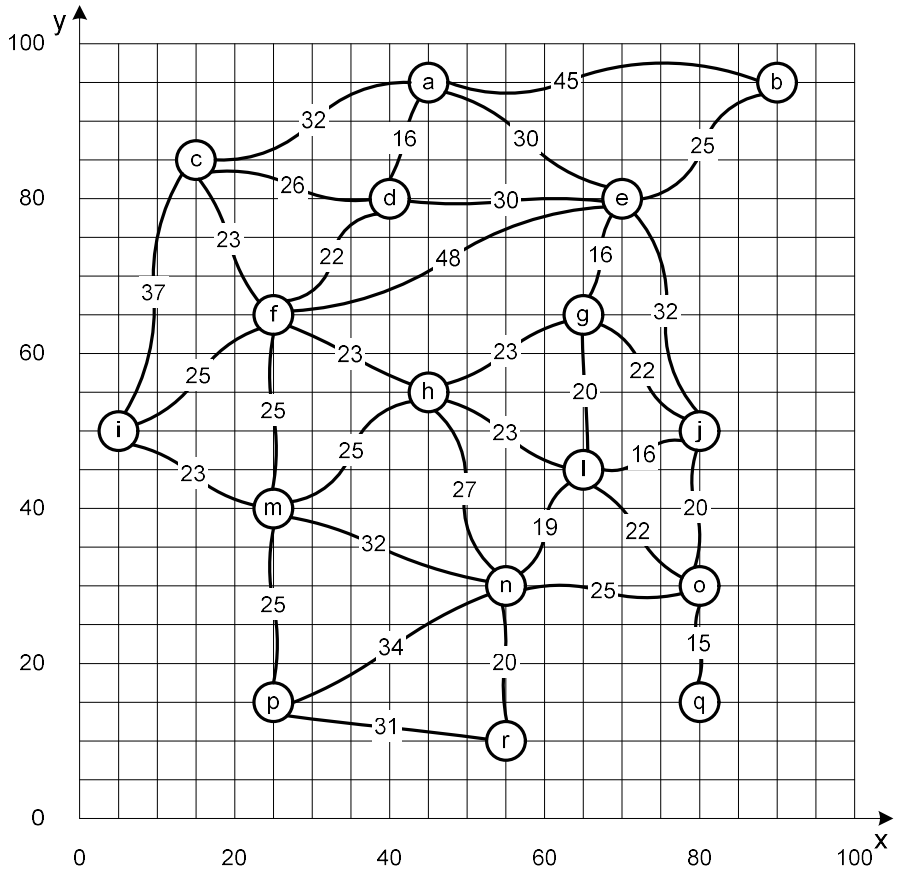
Grafo exemplo

```
%node(Nodo,PosX,PosY)
node(a,45,95).
node(b,90,95).
…
%edge(Nodo1,Nodo2,Custo)
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
…
```



Branch and Bound (Bnb)
Espaço do problema / soluções

```
?- bnb(a,r,Cam,Custo).
Cam = [a, e, g, l, n, r],
Custo = 105 ;
Cam = [a, d, f, h, n, r],
Custo = 108 ;
Cam = [a, d, f, m, n, r],
Custo = 115 ;
Cam = [a, e, g, h, n, r],
Custo = 116 ;
Cam = [a, e, j, l, n, r],
Custo = 117 ;
Cam = [a, d, f, m, p, r],
Custo = 119 ;
Cam = [a, d, e, g, l, n, r],
Custo = 121 ;
Cam = [a, d, f, h, l, n, r],
Custo = 123 ;
…
```



A*

- Interessa, portanto, juntar o que há de bom no Primeiro o Melhor, ou seja o uso de funções que estimam a distância à solução, com o que há de melhor no Branch and Bound, ou seja o uso de custos acumulados conhecidos e a possibilidade de comutar de um ponto para outro na árvore de pesquisa sem que o novo ponto seja um descendente do primeiro. No fundo isso corresponde ao método A*

- No A* consideramos uma função heurística f' tal que:
  - f' = g + h'
    - g é o custo conhecido para ir do estado inicial até ao estado do nó que se está a considerar, e que no momento é uma folha ainda não expandida
    - h' é uma estimativa do custo para ir desse nó até a solução, estando portanto sujeita a erro

A*

- O algoritmo é simples, a cada passo expande-se um nó, gerando mais nós. Os nós que ainda não foram expandidos ficam numa lista, sendo selecionado a partir dessa lista o nó mais promissor, ou seja aquele que apresenta menor valor para f’ se estivermos a trabalhar com custos. O processo termina quando se atinge a primeira solução ou quando na lista não houver nenhum nó mais promissor que uma solução já encontrada

- A primeira parcela de f’, ou seja g, é conhecida, é um custo que já foi despendido. Sobre a segunda parcela, h’, nada sabemos pois é uma estimativa. À medida que nos aproximamos da solução o valor de h’ deve reduzir-se, até que quando se chega à solução h’ deve anular-se.

A*

- O que interessará que h’ seja relativamente ao valor real?

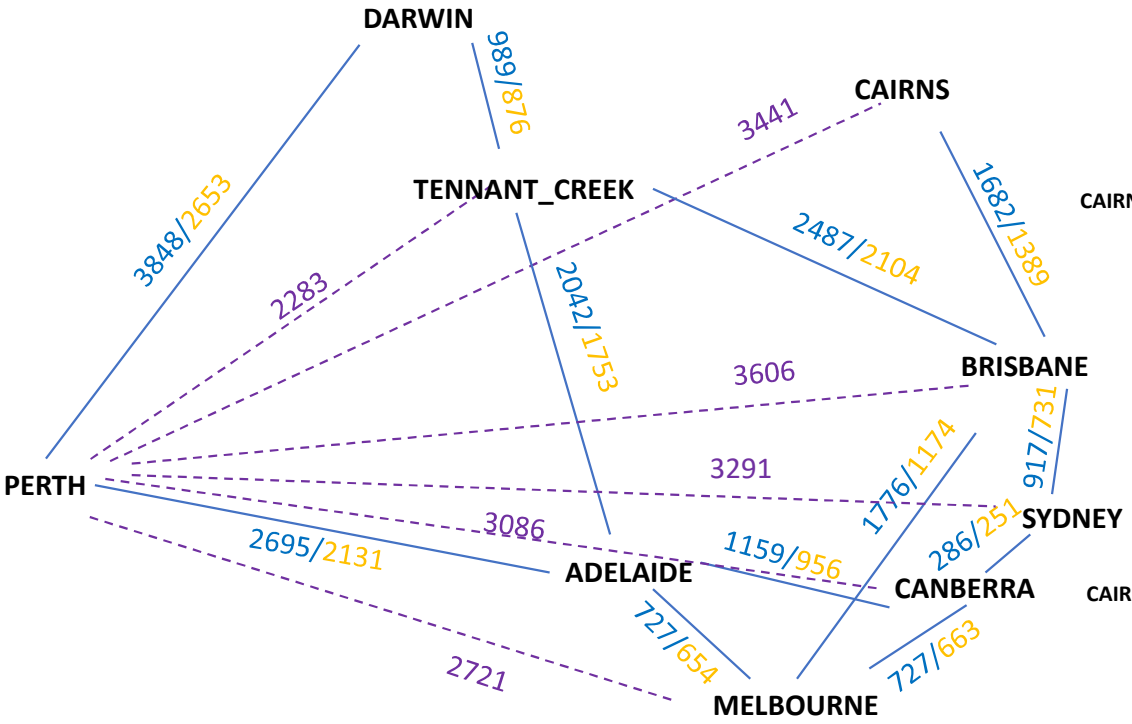
- Seja h’ é uma estimativa da distância de um nó à solução no método de pesquisa A*, se estivermos a operar com custos convém que h’ seja um minorante do custo real, ao passo que se estivermos a operar com lucros interessa que h’ seja um majorante

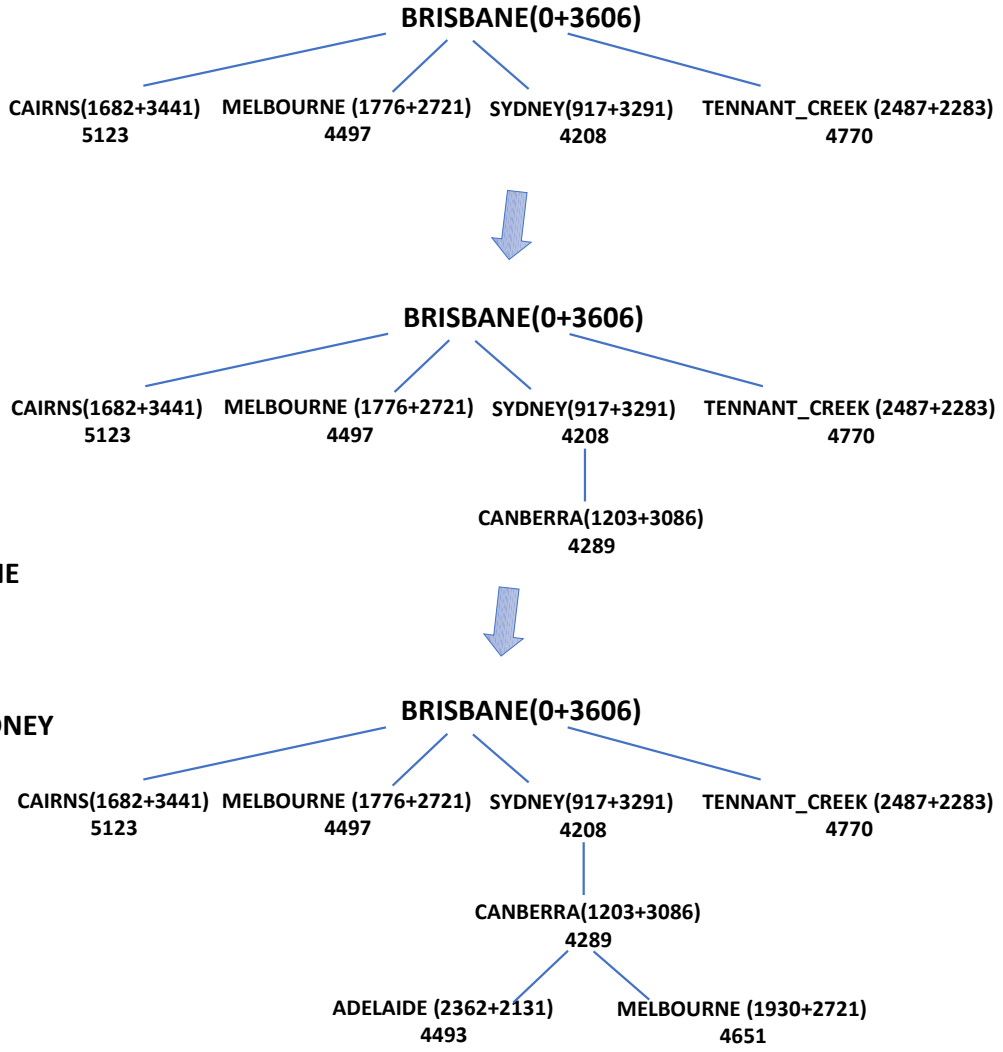
- Porque?

- O que acontece se o minorante for 0?

A*

Objetivo: ir de BRISBANE para PERTH

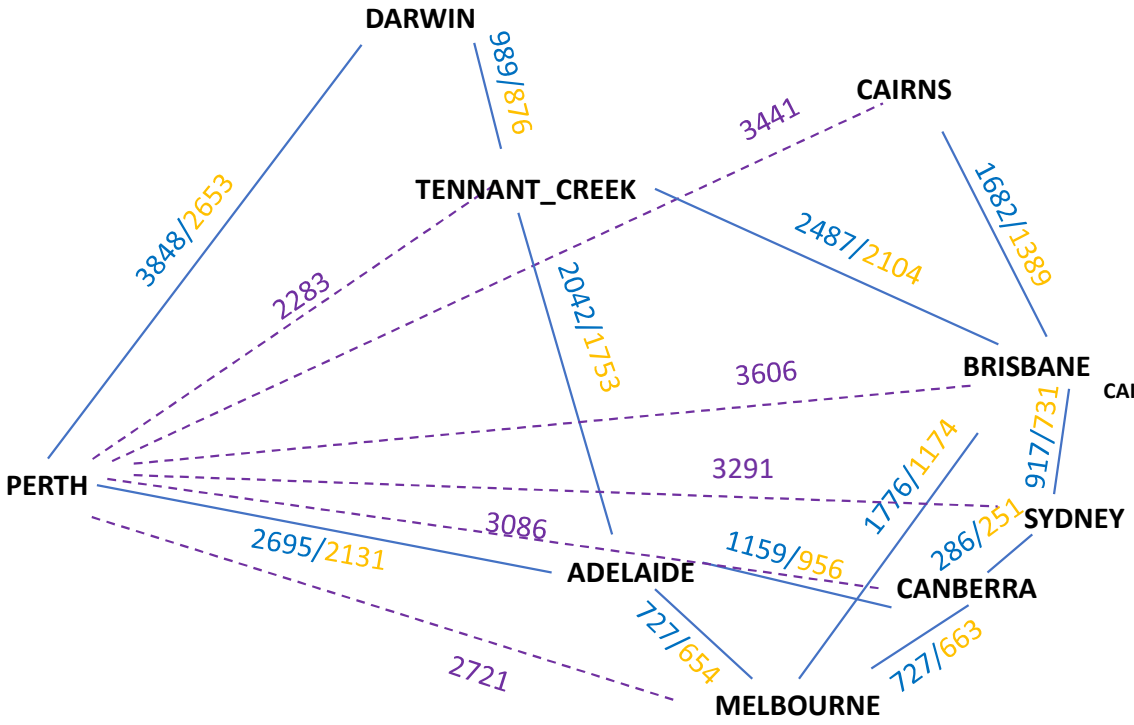


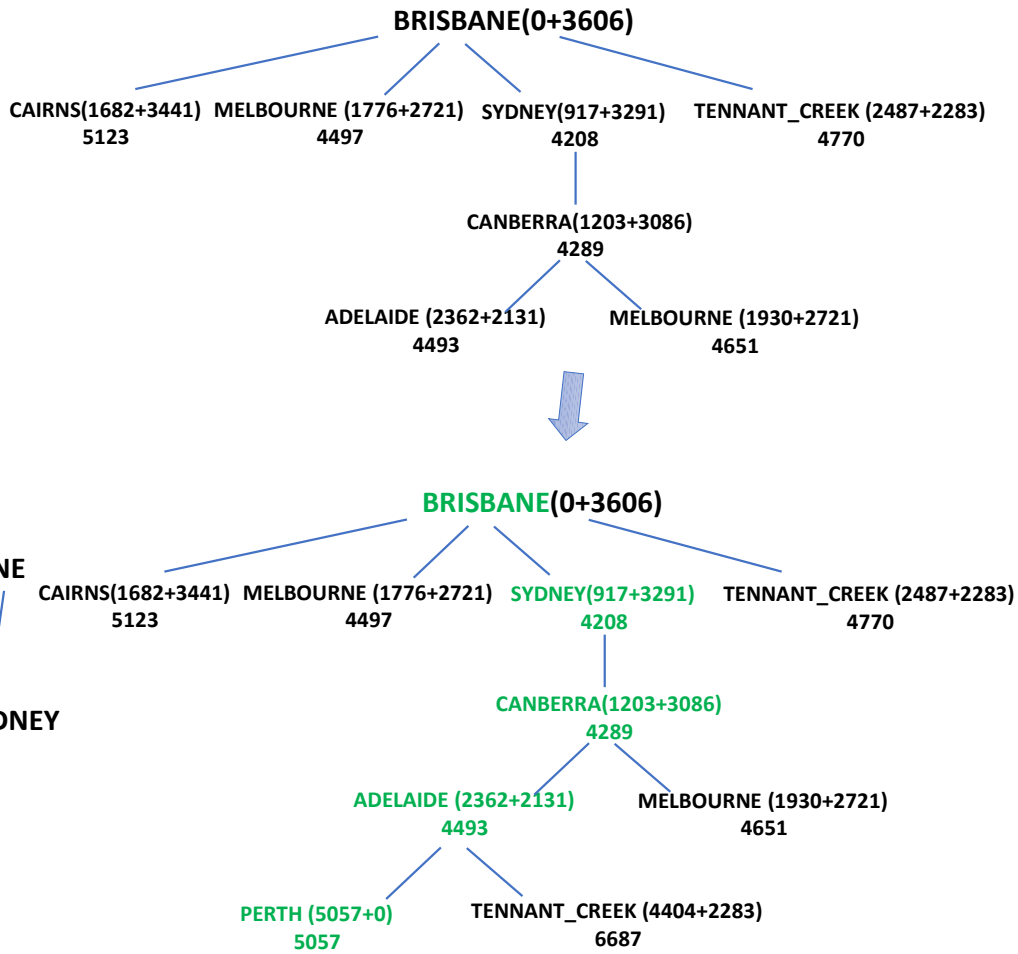


Para este exemplo vamos precisar das distâncias a PERTH

A*

Objetivo: ir de BRISBANE para PERTH





Encontra solução ? Encontra melhor solução ?

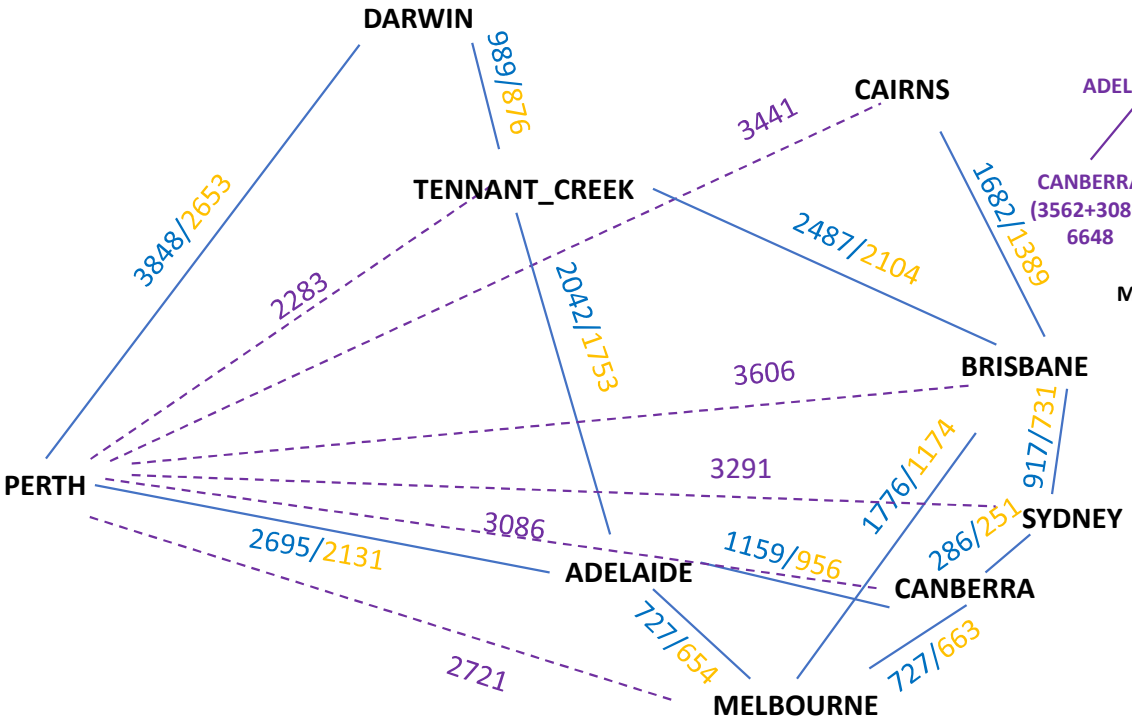
Neste caso temos uma solução mas ainda não podemos garantir que seja a melhor por haver nós folha com valores inferiores

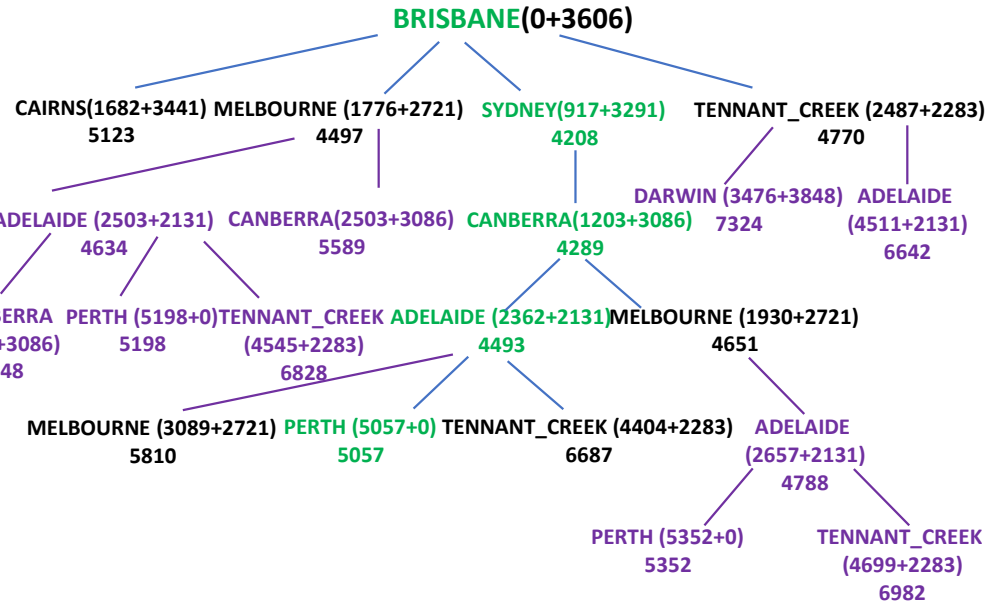
Vamos então continuar até que em todos os nós folha tenhamos somas maiores ou iguais à melhor solução

Mas só podemos fazer isso se a estimativa for um minorante, senão teríamos que comparar só com a primeira parte da soma

A*

Objetivo: ir de BRISBANE para PERTH





E neste momento podemos assegurar que a solução encontrada BRISBANE-SYDNEY-CANBERRA-ADELAIDE-PERTH é de facto a melhor solução para o problema por já não haver nenhum nó folha com valor superior a 5057

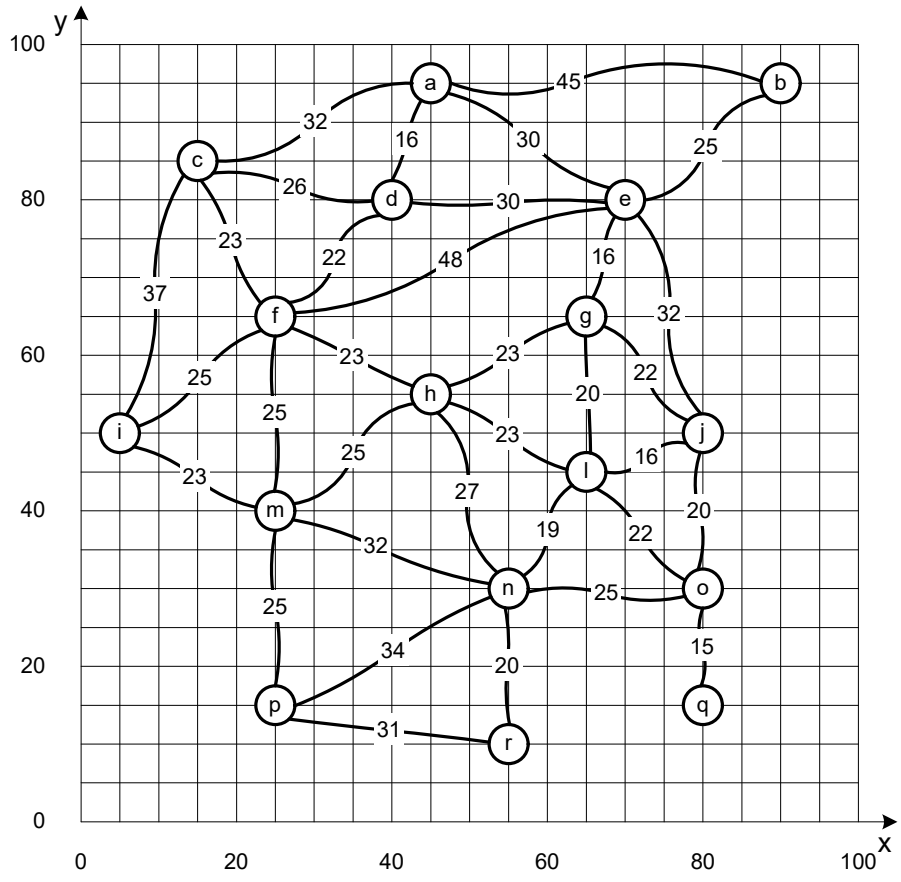
Notar que alguns caminhos seguidos nesta parte final poderiam ser evitados se houvesse algum procedimento de comparação com caminhos anteriormente gerados (por exemplo a exploração de do caminho BRISBANE-SYDNEY-CANBERRA-MELBOURNE com 4651 não faz sentido quando já se explorou o BRISBANE-MELBOURNE com 4497, mas claro isso também demora tempo!

A-Star: Espaço do problema / soluções

```
node(a,45,95).
node(b,90,95).
node(c,15,85).
node(d,40,80).
node(e,70,80).
node(f,25,65).
node(g,65,65).
node(h,45,55).
node(i,5,50).
node(j,80,50).
node(l,65,45).
node(m,25,40).
node(n,55,30).
node(o,80,30).
node(p,25,15).
node(q,80,15).
node(r,55,10).
```

```
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
```

```
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
```



A-Star : Implementação Prolog

```
aStar(Orig,Dest,Cam,Custo):-
      aStar2(Dest,[(_,0,[Orig])],Cam,Custo).

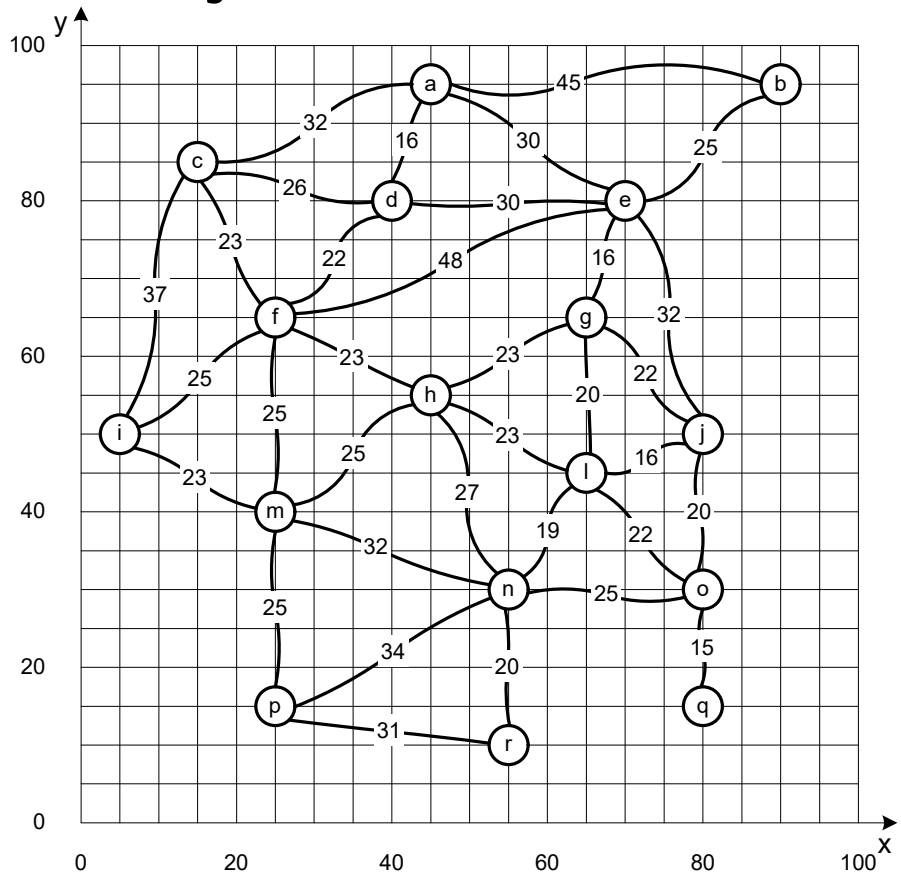
aStar2(Dest,[(_,Custo,[Dest|T])|_],Cam,Custo):-
      reverse([Dest|T],Cam).

aStar2(Dest,[(_,Ca,LA)|Outros],Cam,Custo):-
      LA=[Act|_],
      findall((CEX,CaX,[X|LA]),
            (Dest\==Act,edge(Act,X,CustoX),\+ member(X,LA),
            CaX is CustoX + Ca, estimativa(X,Dest,EstX),
            CEX is CaX +EstX),Novos),
      append(Outros,Novos,Todos),
      sort(Todos,TodosOrd),
      aStar2(Dest,TodosOrd,Cam,Custo).

estimativa(Nodo1,Nodo2,Estimativa):-
      node(Nodo1,X1,Y1),
      node(Nodo2,X2,Y2),
      Estimativa is sqrt((X1-X2)^2+(Y1-Y2)^2).
```

A-Star
Espaço do problema / soluções

```
?- aStar(a,r,Cam,Custo).
Cam = [a, e, g, l, n, r],
Custo = 105 ;
Cam = [a, d, f, h, n, r],
Custo = 108 ;
Cam = [a, d, f, m, n, r],
Custo = 115 ;
Cam = [a, e, g, h, n, r],
Custo = 116 ;
Cam = [a, e, j, l, n, r],
Custo = 117 ;
Cam = [a, d, f, m, p, r],
Custo = 119 ;
Cam = [a, d, e, g, l, n, r],
Custo = 121 ;
Cam = [a, d, f, h, l, n, r],
Custo = 123 ;
Cam = [a, e, g, j, l, n, r],
Custo = 123 ;
…
```



A-Star : Implementação Prolog

```
aStar(Orig,Dest,Cam,Custo):-
      aStar2(Dest,[(_,0,[Orig])],Cam,Custo).

aStar2(Dest,[(_,Custo,[Dest|T])|_],Cam,Custo):-
      reverse([Dest|T],Cam).

aStar2(Dest,[(_,Ca,LA)|Outros],Cam,Custo):-
      LA=[Act|_],
      findall((CEX,CaX,[X|LA]),
            (Dest\==Act,edge(Act,X,CustoX),\+ member(X,LA),
            CaX is CustoX + Ca, estimativa(X,Dest,EstX),
            CEX is CaX +EstX),Novos),
      append(Outros,Novos,Todos),
      write('Novos='),write(Novos),nl,
      sort(Todos,TodosOrd),
      write('TodosOrd='),write(TodosOrd),nl,
      aStar2(Dest,TodosOrd,Cam,Custo).

estimativa(Nodo1,Nodo2,Estimativa):-
      node(Nodo1,X1,Y1),
      node(Nodo2,X2,Y2),
      Estimativa is sqrt((X1-X2)^2+(Y1-Y2)^2).
```

?- aStar(a,r,Cam,Custo).

Novos=[(136.92388155425118,45,[b,a]),(117.0,32,[c,a]),(87.58910531638176,16,[d,a]),(101.58910531638176,30,[e,a])]

TodosOrd=[(87.58910531638176,16,[d,a]),(101.58910531638176,30,[e,a]),(117.0,32,[c,a]),(136.92388155425118,45,[b,a])]

Novos=[(117.58910531638176,46,[e,d,a]),(100.64982043070833,38,[f,d,a])]

TodosOrd=[(100.64982043070833,38,[f,d,a]),(101.58910531638176,30,[e,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(136.92388155425118,45,[b,a])]

Novos=[(107.09772228646443,61,[h,f,d,a]),(105.42640687119285,63,[m,f,d,a]),(127.03124237432849,63,[i,f,d,a])]

TodosOrd=[(101.58910531638176,30,[e,a]),(105.42640687119285,63,[m,f,d,a]),(107.09772228646443,61,[h,f,d,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(127.03124237432849,63,[i,f,d,a]),(136.92388155425118,45,[b,a])]

Novos=[(140.64982043070833,78,[f,e,a]),(101.90169943749474,46,[g,e,a]),(109.16990566028302,62,[j,e,a])]

TodosOrd=[(101.90169943749474,46,[g,e,a]),(105.42640687119285,63,[m,f,d,a]),(107.09772228646443,61,[h,f,d,a]),(109.16990566028302,62,[j,e,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(127.03124237432849,63,[i,f,d,a]),(136.92388155425118,45,[b,a]),(140.64982043070833,78,[f,e,a])]

Novos=[(115.09772228646443,69,[h,g,e,a]),(102.4005494464026,66,[l,g,e,a]),(115.16990566028302,68,[j,g,e,a])]

TodosOrd=[(102.4005494464026,66,[l,g,e,a]),(105.42640687119285,63,[m,f,d,a]),(107.09772228646443,61,[h,f,d,a]),(109.16990566028302,62,[j,e,a]),(115.09772228646443,69,[h,g,e,a]),(115.16990566028302,68,[j,g,e,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(127.03124237432849,63,[i,f,d,a]),(136.92388155425118,45,[b,a]),(140.64982043070833,78,[f,e,a])]

Novos=[(105.0,85,[n,l,g,e,a]),(120.01562118716424,88,[o,l,g,e,a])]

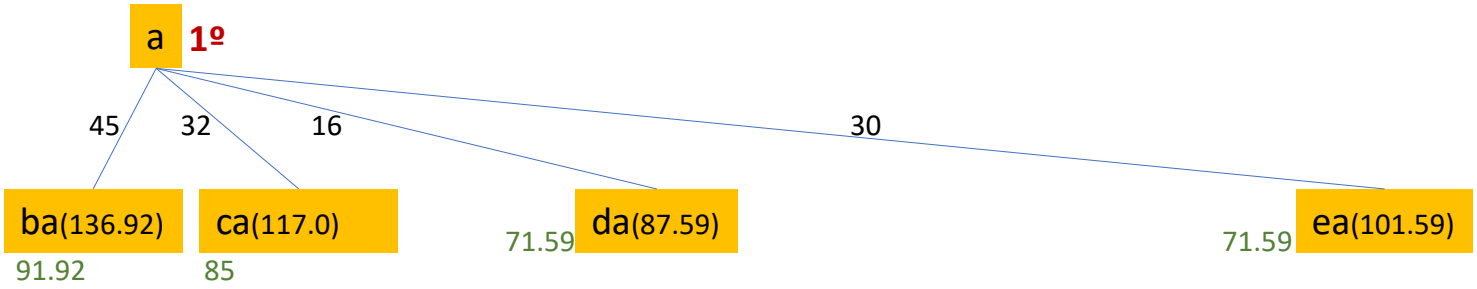
TodosOrd=[(105.0,85,[n,l,g,e,a]),(105.42640687119285,63,[m,f,d,a]),(107.09772228646443,61,[h,f,d,a]),(109.16990566028302,62,[j,e,a]),(115.09772228646443,69,[h,g,e,a]),(115.16990566028302,68,[j,g,e,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(120.01562118716424,88,[o,l,g,e,a]),(127.03124237432849,63,[i,f,d,a]),(136.92388155425118,45,[b,a]),(140.64982043070833,78,[f,e,a])]

Novos=[(149.4138126514911,119,[p,n,l,g,e,a]),(105.0,105,[r,n,l,g,e,a])]

TodosOrd=[(105.0,105,[r,n,l,g,e,a]),(105.42640687119285,63,[m,f,d,a]),(107.09772228646443,61,[h,f,d,a]),(109.16990566028302,62,[j,e,a]),(115.09772228646443,69,[h,g,e,a]),(115.16990566028302,68,[j,g,e,a]),(117.0,32,[c,a]),(117.58910531638176,46,[e,d,a]),(120.01562118716424,88,[o,l,g,e,a]),(127.03124237432849,63,[i,f,d,a]),(136.92388155425118,45,[b,a]),(140.64982043070833,78,[f,e,a]),(149.4138126514911,119,[p,n,l,g,e,a])]

Cam = [a, e, g, l, n, r],

Custo = 105



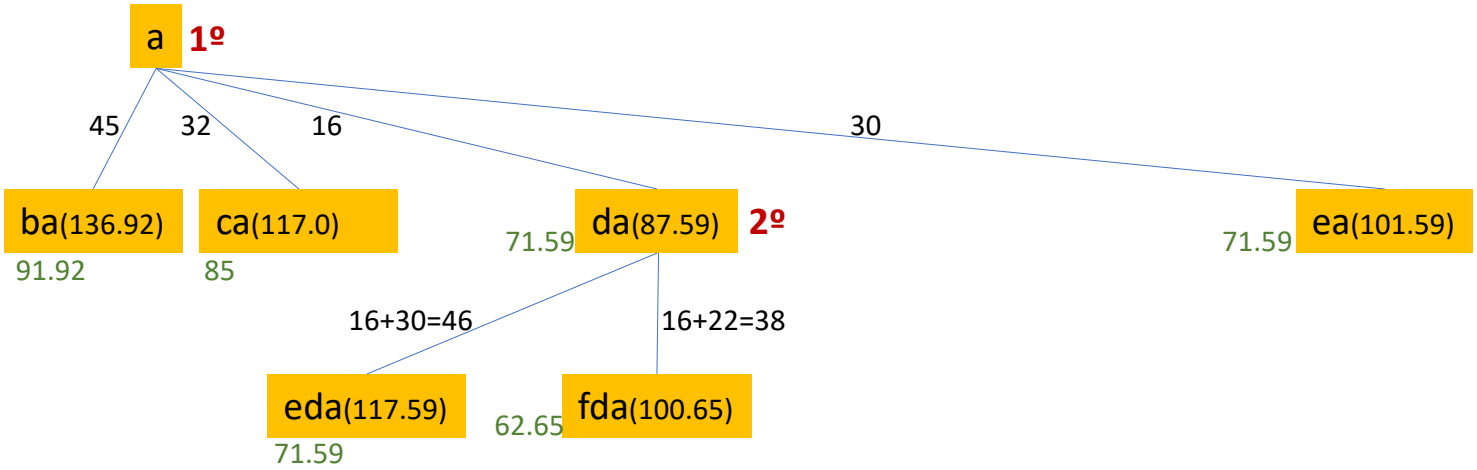
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).

CaX: custo até ao nó atual

caminho invertido(CEX) **NºExpansão**

EstX: estimativa do nó até destino

CEX ← CaX + EstX



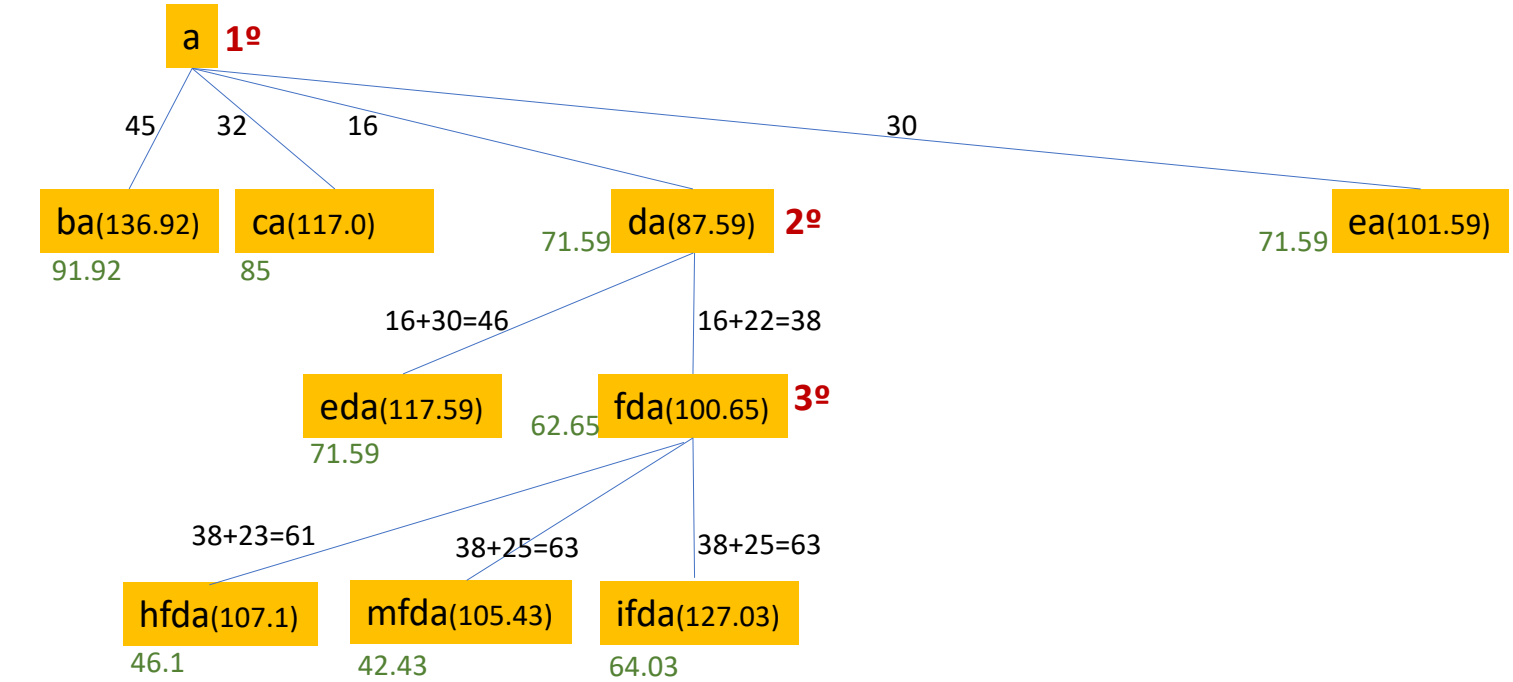

```
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
```

CaX: custo até ao nó atual

caminho invertido(CEX) **NºExpansão**

EstX: estimativa do nó até destino

CEX ← CaX + EstX



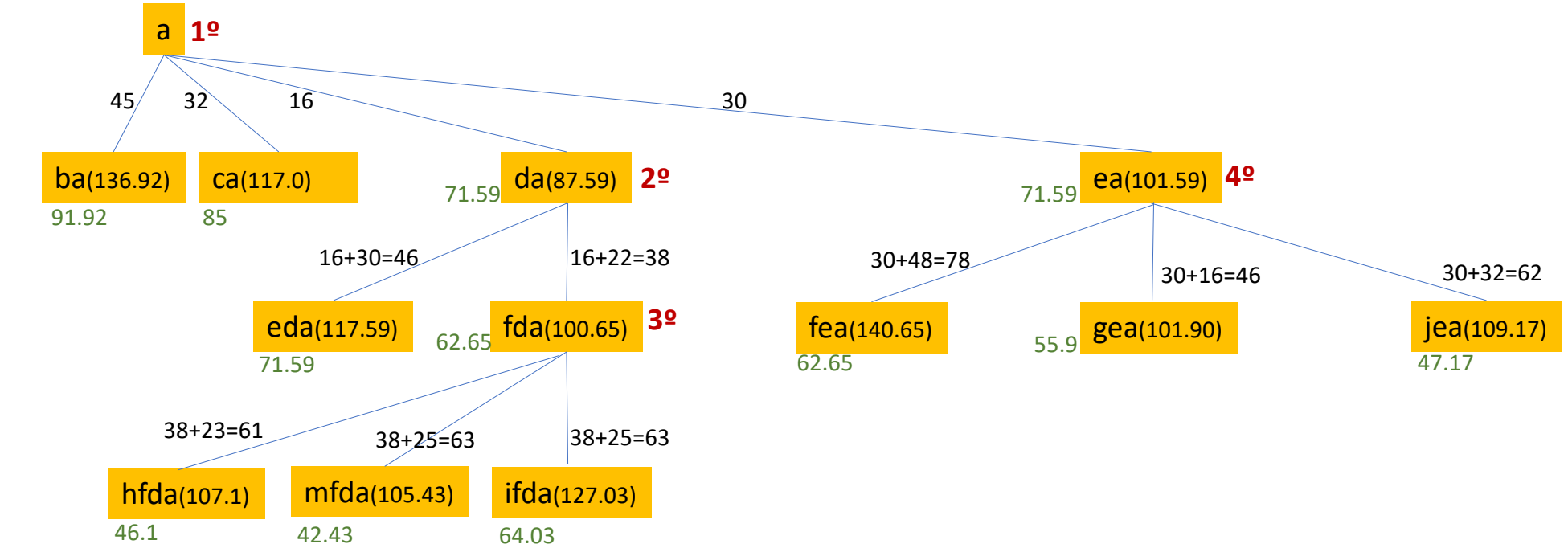
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).

CaX: custo até ao nó atual

caminho invertido(CEX) **NºExpansão**

EstX: estimativa do nó até destino

CEX ← CaX + EstX



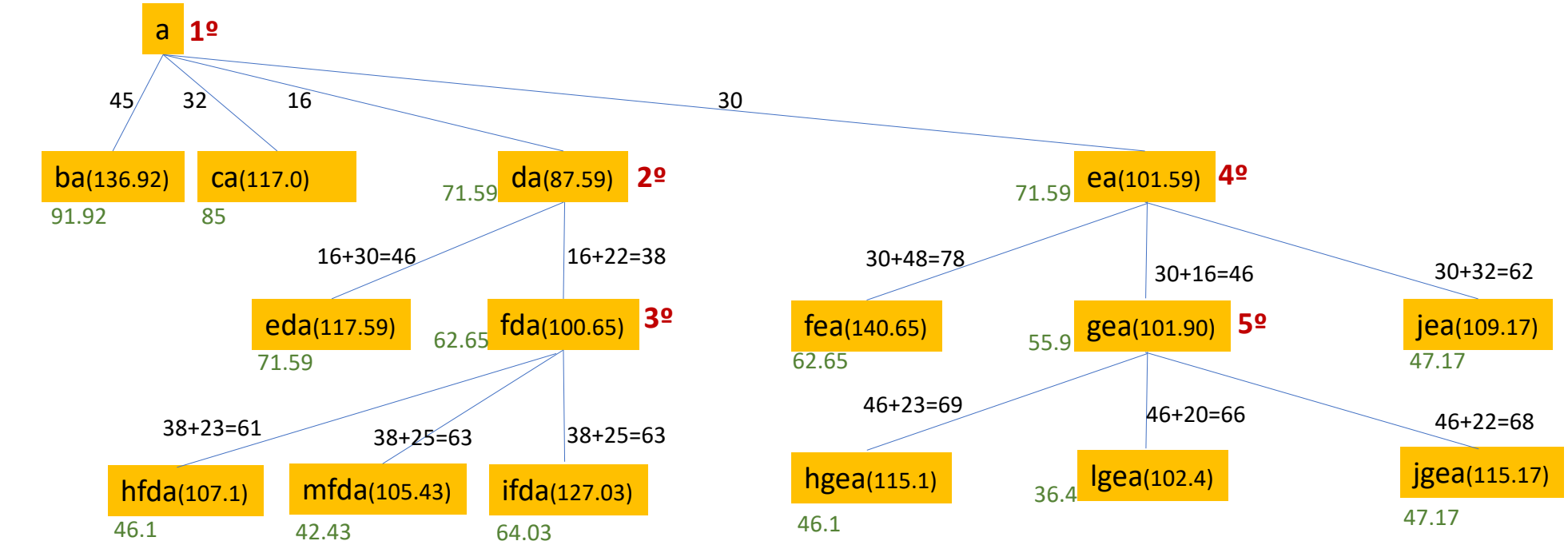
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).

CaX: custo até ao nó atual

caminho invertido(CEX) **NºExpansão**

EstX: estimativa do nó até destino

CEX ← CaX + EstX



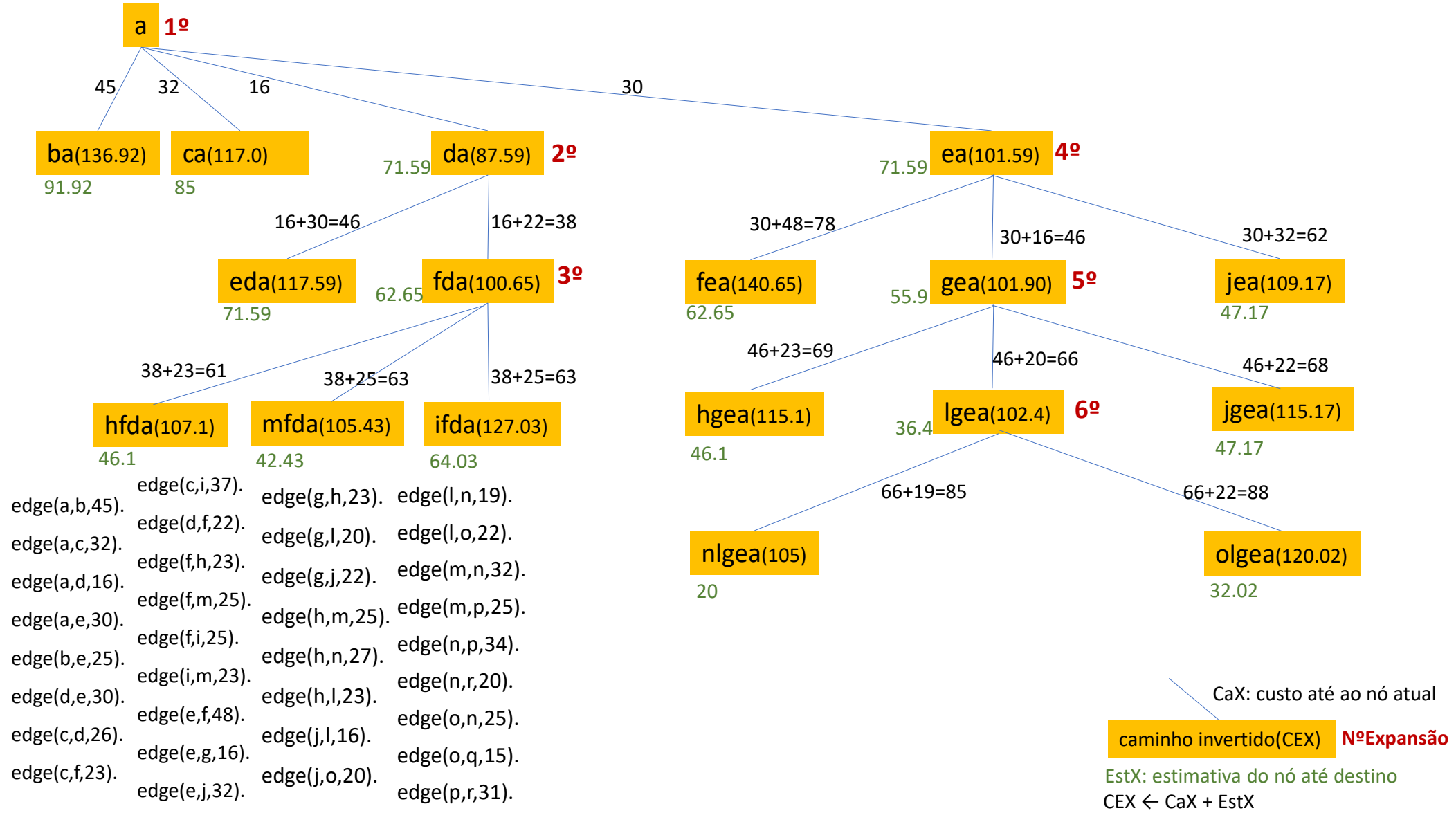

```
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
```

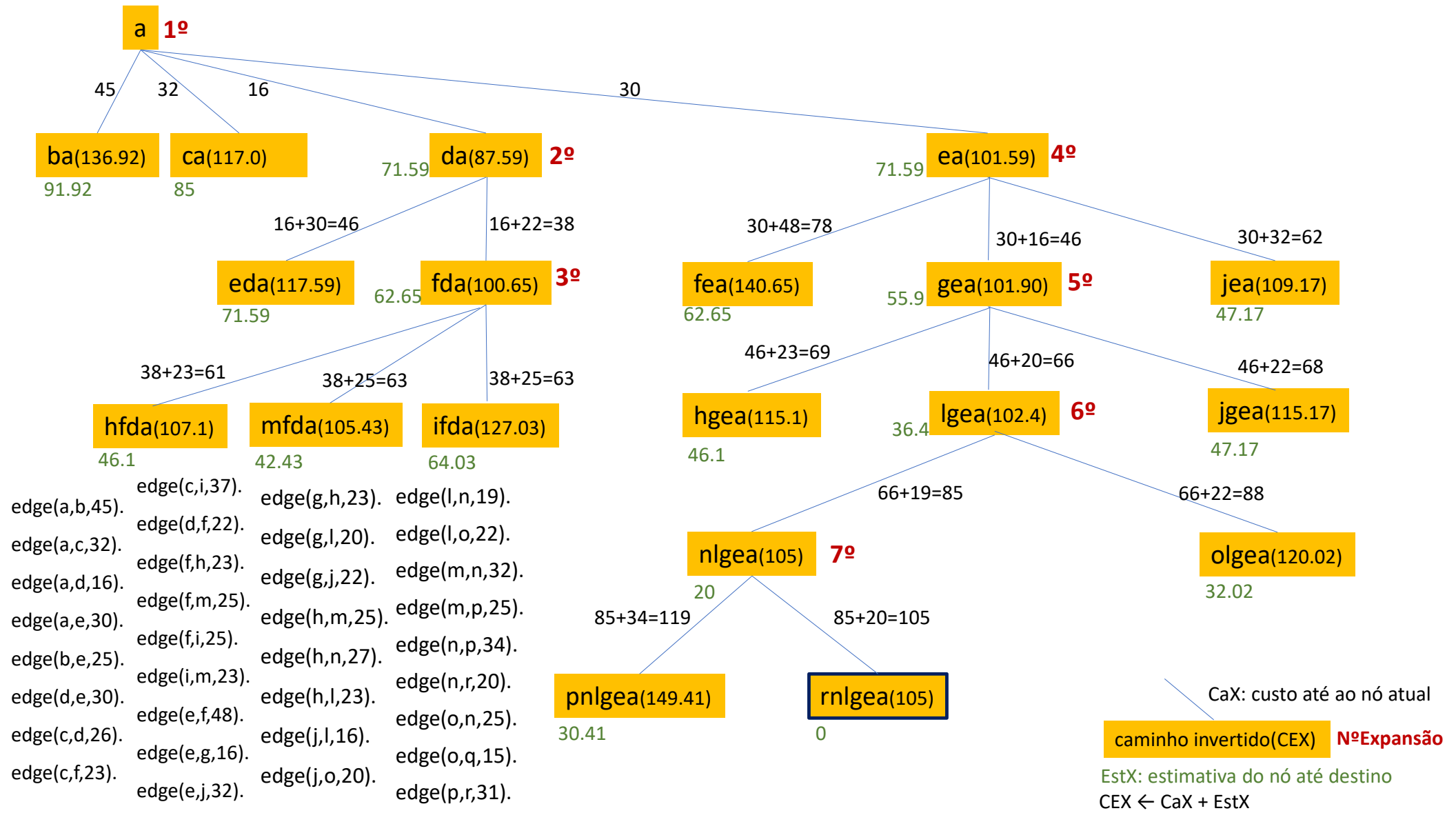
CaX: custo até ao nó atual

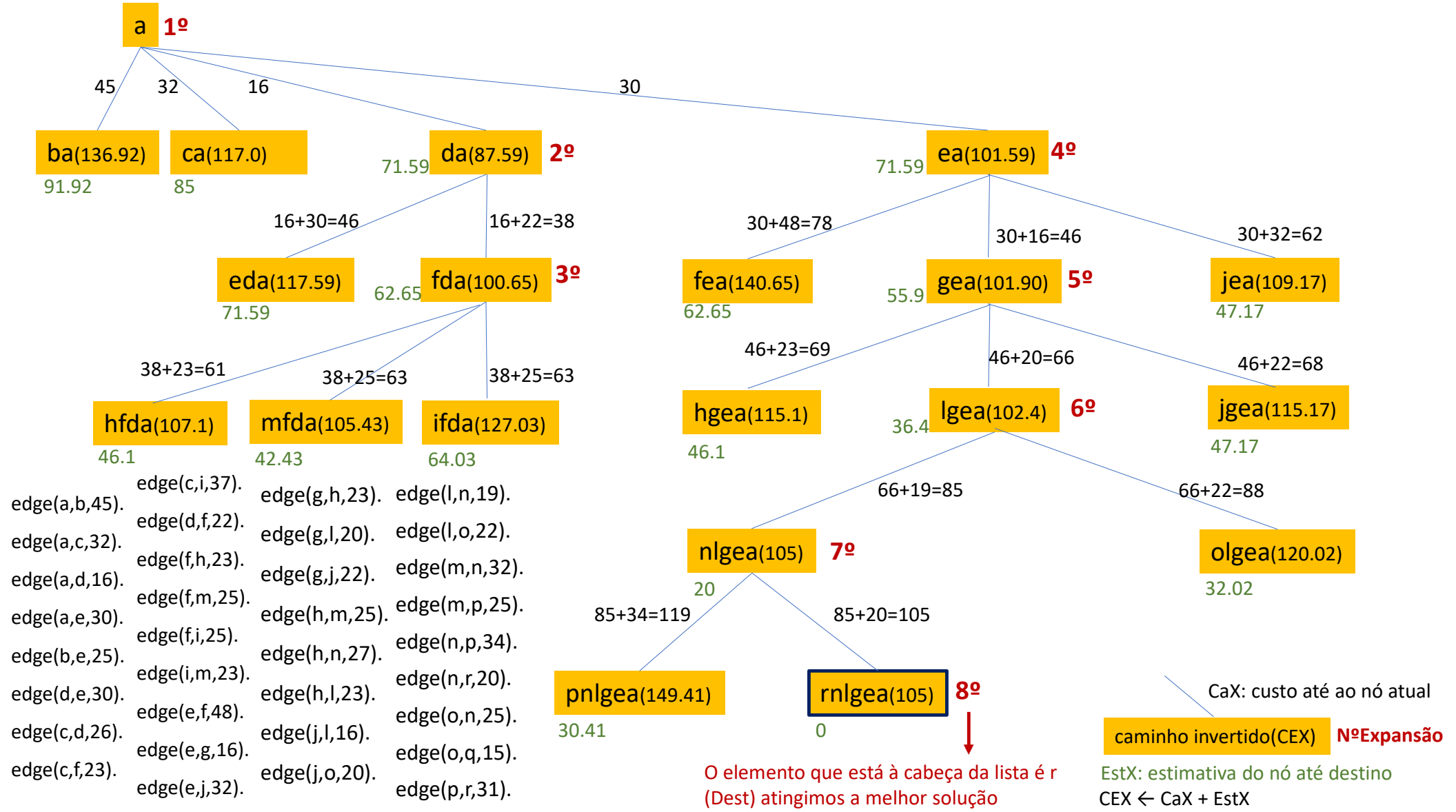
caminho invertido(CEX) **NºExpansão**

EstX: estimativa do nó até destino

CEX ← CaX + EstX


a 1º
45
32
16
30
ba(136.92)
91.92
ca(117.0)
85
71.59
da(87.59) 2º
71.59
ea(101.59) 4º
16+30=46
16+22=38
30+48=78
30+16=46
30+32=62
eda(117.59)
71.59
62.65
fda(100.65) 3º
fea(140.65)
62.65
55.9
gea(101.90) 5º
jea(109.17)
47.17
38+23=61
38+25=63
38+25=63
46+23=69
46+20=66
46+22=68
hfda(107.1)
46.1
mfda(105.43)
42.43
ifda(127.03)
64.03
hgea(115.1)
46.1
36.4
lgea(102.4) 6º
jgea(115.17)
47.17
66+19=85
66+22=88
nlgea(105)
20
olgea(120.02)
32.02
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
CaX: custo até ao nó atual
caminho invertido(CEX)
NºExpansão
EstX: estimativa do nó até destino
CEX ← CaX + EstX


a 1º
45
32
16
30
ba(136.92)
91.92
ca(117.0)
85
71.59
da(87.59) 2º
71.59
ea(101.59) 4º
16+30=46
16+22=38
30+48=78
30+16=46
30+32=62
eda(117.59)
71.59
62.65
fda(100.65) 3º
fea(140.65)
62.65
55.9
gea(101.90) 5º
jea(109.17)
47.17
38+23=61
38+25=63
38+25=63
46+23=69
46+20=66
46+22=68
hfda(107.1)
46.1
mfda(105.43)
42.43
ifda(127.03)
64.03
hgea(115.1)
46.1
36.4
lgea(102.4) 6º
jgea(115.17)
47.17
66+19=85
66+22=88
nlgea(105) 7º
20
olgea(120.02)
32.02
85+34=119
85+20=105
pnlgea(149.41)
30.41
rnlgea(105)
0
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
CaX: custo até ao nó atual
caminho invertido(CEX)
NºExpansão
EstX: estimativa do nó até destino
CEX ← CaX + EstX


a 1º
45
32
16
30
ba(136.92)
91.92
ca(117.0)
85
71.59
da(87.59) 2º
71.59
ea(101.59) 4º
16+30=46
16+22=38
30+48=78
30+16=46
30+32=62
eda(117.59)
71.59
62.65
fda(100.65) 3º
fea(140.65)
62.65
55.9
gea(101.90) 5º
jea(109.17)
47.17
38+23=61
38+25=63
38+25=63
46+23=69
46+20=66
46+22=68
hfda(107.1)
46.1
mfda(105.43)
42.43
ifda(127.03)
64.03
hgea(115.1)
46.1
36.4
lgea(102.4) 6º
jgea(115.17)
47.17
66+19=85
66+22=88
nlgea(105) 7º
20
olgea(120.02)
32.02
85+34=119
85+20=105
pnlgea(149.41)
30.41
rnlgea(105) 8º
0
edge(a,b,45).
edge(a,c,32).
edge(a,d,16).
edge(a,e,30).
edge(b,e,25).
edge(d,e,30).
edge(c,d,26).
edge(c,f,23).
edge(c,i,37).
edge(d,f,22).
edge(f,h,23).
edge(f,m,25).
edge(f,i,25).
edge(i,m,23).
edge(e,f,48).
edge(e,g,16).
edge(e,j,32).
edge(g,h,23).
edge(g,l,20).
edge(g,j,22).
edge(h,m,25).
edge(h,n,27).
edge(h,l,23).
edge(j,l,16).
edge(j,o,20).
edge(l,n,19).
edge(l,o,22).
edge(m,n,32).
edge(m,p,25).
edge(n,p,34).
edge(n,r,20).
edge(o,n,25).
edge(o,q,15).
edge(p,r,31).
O elemento que está à cabeça da lista é r
(Dest) atingimos a melhor solução
CaX: custo até ao nó atual
caminho invertido(CEX)
NºExpansão
EstX: estimativa do nó até destino
CEX ← CaX + EstX

Estimativa

- Se a estimativa for 0 o método A* degenera no Branch&Bound
- Basta substituir estimativa/3 por:

```
estimativa(_,_,0).
```

- O método A* é mais eficiente que o B&B, mas implica haver um modo de calcular a estimativa
- O que interessa que a estimativa seja:
  - Um minorante se estivermos a tratar de um problema de minimização (um majorante se for um problema de maximização)
  - Tão próxima quanto possível do valor real que teremos se seguirmos pelo melhor caminho que leve a solução ótima (se fosse igual a esse valor o método seria imediato, era conduzido diretamente para a solução ótima)

Algoritmos Genéticos

- No início da década 70 John Holland desenvolveu pesquisas pretendendo verificar se aspectos da evolução natural poderiam ser incorporados em algoritmos que permitissem a resolução automática de problemas
- Segundo John Holland, cada solução para um problema poderia ser entendida como um indivíduo de uma população, sendo assim todos os indivíduos da população equivaleriam a um conjunto de soluções possíveis para um problema, cada indivíduo seria representado pelo seu genótipo, ou seja, os indivíduos seriam representados pelos cromossomas
- Tais populações evoluiriam geralmente com base no processo de reprodução, admitindo-se que ocasionalmente pudessem ocorrer mutações
- Desse modo a população ia renovando-se através de gerações, a que corresponderiam ciclos do algoritmo
- O processo repetia-se até que chegássemos a um indivíduo (solução) que apresentasse as características mínimas pretendidas ou até que um limite temporal impusesse o final do algoritmo

Algoritmos Genéticos

Evolução Natural (Genética)	Algoritmos Genéticos
Indivíduo	Solução do Problema
Gene	Parte de uma Solução
Genótipo – Cromossoma – Sequência de Genes	Representação de uma solução – sequência de partes
Reprodução – Combina Cromossomas dos Pais	Seleção + Operador de Cruzamento: gera descendentes
Mutação – afeta Genes no Cromossoma	Operador de Mutação sobre alguns descendentes
População	Conjunto de Soluções, incluindo soluções descendentes
Geração	Soluções que passam para a iteração seguinte

Algoritmos Genéticos

O algoritmo que se segue sumaria o funcionamento de um algoritmo genético típico:

Gerar população inicial de soluções (geração inicial)

Avaliar as soluções da geração inicial

Enquanto o critério de paragem não for atendido

- Selecionar alguns indivíduos da geração para efetuar cruzamento gerando os descendentes
- Aplicar a mutação a alguns dos descendentes
- Avaliar os descendentes depois dos cruzamentos e mutações
- Criar nova população a partir da geração mais os seus descendentes, removendo repetições de soluções
- Selecionar os elementos dessa nova população que passam para a geração seguinte

Fim

Algoritmos Genéticos - Operadores

- O operador de cruzamento combina as características de dois indivíduos (progenitores) de modo a gerar normalmente dois novos indivíduos (descendentes), obtidos por composição do material genético dos seus progenitores
- O operador de mutação consiste na alteração arbitrária de 1 ou mais genes de um cromossoma, com uma probabilidade arbitrária. Este operador garante a existência de alguma diversidade genética na população, pretendendo prevenir a convergência prematura para ótimos locais, através da amostragem aleatória de novos pontos no espaço de pesquisa
- Ao cruzar e mutar ciclicamente e selecionar os “indivíduos” que passam para a próxima geração implementamos os Algoritmos Genéticos

Algoritmos Genéticos - Cruzamento

- O operador principal dos Algoritmos Genéticos é o operador de cruzamento
- Dois indivíduos progenitores são escolhidos de entre os indivíduos da população através de um método de seleção bem definido
- Quando os cromossomas são representados apenas por sequências (por exemplo, de bits, números ou caracteres), o operador de cruzamento produz geralmente os dois descendentes pela escolha de um ou mais pontos de corte nos cromossomas dos progenitores e depois efetua a mistura das partes resultantes para gerar cada um dos cromossomas dos descendentes
- Alternativamente, os descendentes podem ser gerados através de máscaras que indicam se o gene de cada posição deve ser tomado de um dos progenitores ou do outro
- Há portanto 3 alternativas básicas de cruzamento:
  - cruzamento num ponto de corte
  - cruzamento em dois pontos de corte
  - cruzamento uniforme (com máscara)
- Por vezes o cruzamento não origina soluções viáveis, nessas situações há que repensar o modo de efetuar o cruzamento, um exemplo é o cruzamento com manutenção de ordem (order crossover)

Algoritmos Genéticos - Cruzamento

- O operador de cruzamento num ponto faz a seleção aleatória de uma posição de corte
- O cromossoma do descendente D1 é obtido pela parte do cromossoma que está antes do ponto de corte no progenitor P1 e pela parte do cromossoma do progenitor P2 que está depois do ponto de corte
- O cromossoma do descendente D2 é obtido pela parte do cromossoma que está antes do ponto de corte no progenitor P2 e pela parte do cromossoma do progenitor P1 que está depois do ponto de corte

Algoritmos Genéticos - Cruzamento

- Vejamos um exemplo considerando a codificação binária para os cromossomas dos progenitores P1 e P2:
  P1 = 10110010
  P2 = 11010110
- Se considerarmos o ponto de corte entre o 5º e o 6º genes (ou bit, da esquerda para a direita) então os cromossomas dos progenitores ficam cortados da seguinte maneira:
  P1 = 10110-010
  P2 = 11010-110
- Originando os seguintes descendentes:
  D1= 10110-110 = 10110110
  D2= 11010-010 = 11010010

Algoritmos Genéticos - Cruzamento

- O operador de cruzamento em dois pontos de corte selecciona aleatoriamente dois pontos de corte, dividindo os cromossomas de cada progenitor em 3 partes
- O cromossoma do primeiro descendente é obtido pelas partes extremas do cromossoma do primeiro progenitor e pela parte central do cromossoma do segundo progenitor
- O cromossoma do segundo descendente é obtido pelas partes extremas do cromossoma do segundo progenitor e pela parte central do cromossoma do primeiro progenitor

Algoritmos Genéticos - Cruzamento

- Vamos reconsiderar os progenitores P1 e P2 da secção anterior:
  P1 = 10110010
  P2 = 11010110
- Admita-se que os pontos de corte situam-se agora entre o 2º e 3º genes e entre o 5º e 6º genes teremos os seguintes cortes nos progenitores:
  P1 = 10-110-010
  P2 = 11-010-110
- Agora os descendentes são obtidos da seguinte forma:
  D1 = 10-010-010 = 10010010
  D2 = 11-110-110 = 11110110

Algoritmos Genéticos - Cruzamento

- Poderíamos estender o conceito anterior para mais pontos de corte (3, 4, etc)
- Tal é possível com o operador de cruzamento uniforme que vai gerar aleatoriamente as posições dos genes que serão trocadas, o que corresponde a gerar uma máscara binária de igual dimensão a dos cromossomas, significando o 1 que o gene deve ser trocado na posição respetiva e o 0 que o gene não deve ser trocado

Algoritmos Genéticos - Cruzamento

- Vamos tomar novamente os progenitores P1 e P2:

  P1 = 10110010

  P2 = 11010110

- Se a máscara gerada aleatoriamente for a seguinte:

  M = 01110110

- obteremos os seguintes descendentes:

  D1 = 1**10**1**1**0

  D2 = 1**0**11**0**10

Algoritmos Genéticos - Cruzamento

- Outro operador de cruzamento é o “Order Crossover”
- Vamos considerar o exemplo que temos usado das 5 tarefas
- A codificação dos cromossomas poderia ser feita por sequências de 5 elementos do alfabeto: {t1,t2,t3,t4,t5}, por exemplo [t3,t1,t2,t5,t4]

Algoritmos Genéticos - Cruzamento

- Dois potenciais progenitores poderiam ser os seguintes:
  Pa = [t3,t1,t2,t5,t4]
  Pb = [t1,t5,t2,t4,t3]
- Se por ventura adotássemos, por exemplo, o cruzamento em 2 pontos (entre o 1º e 2º genes e entre o 3º e 4º genes) teríamos os seguintes pontos de corte:
  t3 - t1,t2 - t5,t4
  t1 - t5,t2 - t4,t3
- se efetuássemos o cruzamento especificado com 2 pontos de corte teríamos os seguintes descendentes:
  Da = t1 - t1,t2 - t4,t3 = [t1,t1,t2,t4,t3] ← notar que t1 aparece repetido e t5 não aparece
  Db = t3 - t5,t2 - t5,t4 = [t3,t5,t2,t5,t4] ← notar que t5 aparece repetido e t1 não aparece
  portanto os descendentes não são soluções viáveis

Algoritmos Genéticos - Cruzamento

- Por essa razão surgem operadores de cruzamento como o operador de cruzamento com manutenção de ordem (Order Crossover) onde os genes a substituir de um dos progenitores aparecerão na ordem em que estão no outro progenitor. Essa troca poderá proceder-se a partir do primeiro ou do segundo ponto de corte

- Considerando L como a localização de um gene livre e admitindo que a parte central se mantém, teremos para o exemplo citado as seguintes sequências:

  Sequência obtida a partir de Pa = L - t1,t2 – L,L

  Sequência obtida a partir de Pb = L - t5,t2 – L,L

Algoritmos Genéticos - Cruzamento

- Sequência a partir do 2º ponto de corte de Pb = t4,t3,t1,t5,t2
- Sequência a partir do 2º ponto de corte de Pa = t5,t4,t3,t1,t2
- Vamos agora eliminar os elementos conhecidos na parte central do cromossoma de Pa (t1,t2) na sequência a partir do 2º ponto de corte de Pb e os elementos conhecidos na parte central do cromossoma de Pb (t5,t2) na sequência a partir do 2º ponto de corte de Pa, originando as seguintes sequências

  Sequência a partir do 2º ponto de corte de Pb eliminando t1,t2 = t4,t3,~~t1~~,t5,~~t2~~ = t4,t3,t5

  Sequência a partir do 2º ponto de corte de Pa eliminando t5,t2 = ~~t5~~,t4,t3,t1,~~t2~~ = t4,t3,t1
- Tais sequências vão preencher na ordem especificada as posições livres (L) anteriormente referidas, a partir do segundo ponto de corte, originando os descendentes Da e Db que se seguem:

  Da: Sequência obtida a partir de Pa = L - t1,t2 - L,L = t5 - t1,t2 - t4,t3 ← [t5,t1,t2,t4,t3]

  Db: Sequência obtida a partir de Pb = L - t5,t2 - L,L = t1 - t5,t2 - t4,t3 ← [t1,t5,t2,t4,t3]

  Agora os descendentes já são viáveis

Algoritmos Genéticos - Mutação

- O operador de mutação usado nos Algoritmos Genéticos tem o objetivo de trazer de volta para a população os genes perdidos durante o processo de seleção, de modo a que possam ser testados num novo contexto. Serve ainda para proporcionar novos genes que não estavam originalmente na população inicial
- Geralmente, a mutação é efetuada em apenas 1 gene, embora possa ser feita sobre mais genes. Na posição escolhida aleatoriamente o gene do cromossoma é substituído por um outro gene viável para aquela posição, oriundo do património genético da população (conjunto de todos os genes que poderão aparecer na população)

Algoritmos Genéticos - Mutação

- Considere-se, por exemplo, o descendente D:

  D = 10110010

- Se o cromossoma deste descendente for sujeito a uma mutação no 5º gene, que continha um 0, o gene é substituído por um 1 (visto que não existe nenhuma outra alternativa de substituição dado que estamos a tratar de sequências binárias). Sendo assim a nova versão do descendente será:

  D’ = 1011**1**010

Algoritmos Genéticos - Mutação

- Considere-se agora o descendente Db:
  Db = [t1,t5,t2,t4,t3]
- Se aplicássemos a mutação a um dado gene (por exemplo, o 2º gene que tem o valor t5) e originássemos o conjunto dos genes que poderiam aparecer nessa posição obteríamos o seguinte conjunto: {t1,t2,t3,t4}
- Observe-se que da mutação do gene da 2ª posição resultaria sempre uma solução não viável, onde o t5 não apareceria e uma outra tarefa estaria repetida
- Por essa razão a mutação aqui deveria ser efetuada em 2 genes, trocando os respetivos conteúdos. Por exemplo, se trocarmos o 2º e 3º gene de Db obteremos a seguinte mutação:
  Db' = [t1,t2,t5,t4,t3]
- Isso faz lembrar algo (os vizinhos na Pesquisa Local), neste exemplo Db' é um dos vizinhos de ordem 1 de Db pelo facto dos genes trocados serem consecutivos, mas poderia ser de ordem 2 se fossem trocados o 2º e 4º gene

Algoritmos Genéticos – Exemplo 1

- Este primeiro exemplo é simples, mas permitirá verificarmos como pode evoluir um algoritmo genético
- Vamos considerar a função $f(x)=x^2$, com x inteiro e pertencente ao intervalo [0,63]. O objetivo será maximizar o valor da respetiva função
- A codificação das soluções por uma string com 6 dígitos binários é adequada
- Vamos considerar o seguinte quadro que nos dá os elementos da população e o valor da função de avaliação (igual a f(x))

Algoritmos Genéticos – Exemplo 1

- Vamos considerar a seguinte população inicial

Elemento nº	Geração 1	Solução x	f(x)
1	100101	37	1369
2	001001	9	81
3	010011	19	361
4	001100	12	144

Algoritmos Genéticos – Exemplo 1

Vejamos o que acontece após o 1º cruzamento:

Elemento nº	População após cruzamento	Solução x	f(x)
1	100101	37	1369
2	001001	9	81
3	010011	19	361
4	001100	12	144
5 - cruz.1,2	100001	33	1089
6 - cruz.1,2	001101	13	169
7 - cruz.3,4	000110	6	36
8 - cruz.3,4	011001	25	625

Vamos fazer os cruzamento usando o cruzamento uniforme com uma máscara M = 010101

Algoritmos Genéticos – Exemplo 1

São então seleccionados os elementos 1, 3, 5 e 8, que são renumerados, originando a geração 2 que se segue:

Elemento nº	Geração 2	Solução x	f(x)
1	100101	37	1369
2	010011	19	361
3	100001	33	1089
4	011001	25	625

Algoritmos Genéticos – Exemplo 1

Vamos repetir o ciclo de cruzamento mais uma vez:

Elemento nº	População após cruzamento	Solução x	f(x)
1	100101	37	1369
2	010011	19	361
3	100001	33	1089
4	011001	25	625
5 - cruz.1,2	110001	49	2401
6 - cruz.1,2	000111	7	49
7 - cruz.3,4	110001	49	2401
8 - cruz.3,4	001001	9	81

Algoritmos Genéticos – Exemplo 1

Agora são seleccionados os elementos 1, 3, 4 e 5, que originam a geração nº 3:

Elemento nº	Geração 3	Solução x	f(x)
1	100101	37	1369
2	100001	33	1089
3	011001	25	625
4	110001	49	2401

Algoritmos Genéticos – Exemplo 1

Não iremos continuar exemplificando as gerações seguintes. Vamos apenas observar alguns aspetos:

Em primeiro lugar, e após 2 ciclos de cruzamento foi possível verificar que a população tem fatores de avaliação superiores que a população original e que o melhor elemento da população tem um fator de avaliação superior

Quererá isso dizer que rapidamente convergiremos para a solução ótima?

- Infelizmente, esse não é o caso. Se continuarmos a aplicar apenas cruzamentos a partir da geração 3, até poderemos garantir que a melhor solução nunca será encontrada. A melhor solução é 111111 e note-se que no 5º bit, da esquerda para a direita todos os elementos da população têm o valor 0
- Daí o interesse das mutações!

Algoritmos Genéticos – Exemplo 2

Vamos tomar como base um problema diferente do nosso, mas que também é um derivado do tradicional problema do Caixeiro Viajante – o problema do Escalonamento de tarefas numa máquina

Por exemplo, escalonar 5 tarefas numa máquina

- Para cada tarefa **j** (j=1, …, 5),
  - $\mathbf{p_j}$ é o tempo de processamento,
  - $\mathbf{d_j}$ é o tempo limite de conclusão
  - $\mathbf{w_j}$ é o peso da penalização em caso de atraso
- Objetivo: minimizar a soma pesada dos atrasos $\mathbf{\Sigma w_j T_j}$, onde $\mathbf{T_j}$ é o atraso da tarefa j

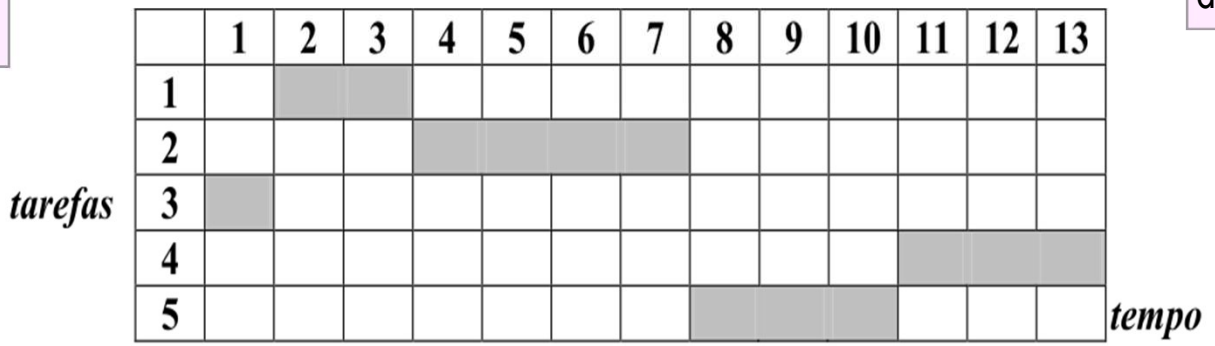
Algoritmos Genéticos – Exemplo 2

Para visualizar uma sequência é muitas vezes utilizado um diagrama de Gantt *tarefas x tempo*

Para a sequência de tarefas [3,1,2,5,4] obtém-se o calendário representado a seguir

Tempo de Processamento
p1=2
p2=4
p3=1
p4=3
p5=3

Conclusão
d1=5
d2=7
d3=11
d4=9
d5=8



Algoritmos Genéticos – Exemplo 2

Diagrama GANTT visualizando uma sequência possível [t3,t1,t2,t5,t4]

Processamento	Conclusão	Pesos
p1=2	d1=5	w1=1
p2=4	d2=7	w2=6
p3=1	d3=11	w3=2
p4=3	d4=9	w4=3
p5=3	d5=8	w5=2

	1	2	3	4	5	6	7	8	9	10	11	12	13
1													
2													
3													
4													
5													

$\Sigma w_j T_j$ = 0+0+0+(13-9)x3+(10-8)x2=4x3+2x2=16

Algoritmos Genéticos – Exemplo 2

- Vamos agora assumir uma população inicial

n. Solução	Solução	Avaliação
1	[t3, t1, t2, t5, t4]	16
2	[t5, t1, t2, t4, t3]	25
3	[t3, t5, t2, t4, t1]	20
4	[t4, t3, t2, t1, t5]	21

Algoritmos Genéticos – Exemplo 2

n. Solução	Solução	Avaliação
1	[t3, t1, t2, t5, t4]	16
2	[t5, t1, t2, t4, t3]	25
3	[t3, t5, t2, t4, t1]	20
4	[t4, t3, t2, t1, t5]	21
5 (Cruz. 1 + 2) corte 2-4	[t4, t1, t2, t5, t3]	24
6 (Cruz. 1 + 2) corte 2-4	[t5, t1, t2, t4, t3]	25
7 (Cruz. 3 + 4) corte 1-3	[t3, t5, t2, t1, t4]	23
8 (Cruz. 3 + 4) Corte 1-3 Mut 2-5	[t4, t5, t2, t1, t3]	29

Estamos a usar o Order Crossover para o cruzamento e a mutação por troca de posição Entre 2 genes

Algoritmos Genéticos – Exemplo 2

n. Solução	Solução	Avaliação
1	[t3, t1, t2, t5, t4]	16
3	[t3, t5, t2, t4, t1]	20
4	[t4, t3, t2, t1, t5]	21
7	[t3, t5, t2, t1, t4]	23
9 (Cruz. 1 + 4) corte 3-5	[t3, t1, t2, t5, t4]	16
10 (Cruz. 1 + 4) corte 3-5 Mut 2,4	[t3, t1, t2, t4, t5]	13
11 (Cruz. 3 + 7) corte 2-4	[t1, t5, t2, t4, t3]	25
12 (Cruz. 3 + 7) corte 2-4	[t4, t5, t2, t1, t3]	29

Algoritmos Genéticos – Exemplo 2

n. Solução	Solução	Avaliação
1	[t3, t1, t2, t5, t4]	16
3	[t3, t5, t2, t4, t1]	20
4	[t4, t3, t2, t1, t5]	21
10	[t3, t1, t2, t4, t5]	13
…	…	…
…	…	…
…	…	…
…	…	…

Temos um valor já melhor que o melhor da população inicial

A solução ótima para este exemplo é [t1,t2,t4,t3,t5] com custo 10

Algoritmos Genéticos – Parametrização

Os principais parâmetros de um Algoritmo Genético são:

- A dimensão da população/gerações
- a taxa de cruzamento
- a taxa de mutação
- a taxa de substituição
- os critérios de paragem

Algoritmos Genéticos – Parametrização Dimensão da População/Gerações

O tamanho da população/gerações condiciona a qualidade da solução obtida e o tempo de processamento

Se a população for muito pequena leva a que provavelmente tenhamos uma fraca diversidade das soluções que podem ser obtidas no espaço de pesquisa, geralmente chega-se rapidamente a uma situação na qual a população não é melhorada, a não ser que casualmente as mutações originem indivíduos mais interessantes

Se a população for muito grande, temos uma melhor cobertura do espaço de pesquisa, e a probabilidade da população estabilizar um máximos locais é menor. Contudo, tal é conseguido com custos computacionais

Geralmente, os algoritmos genéticos admitem que a população mantém-se constante, embora nada impeça a sua variação dinâmica, embora em termos práticos se deva ter cuidado em evitar a diminuição ou crescimento abrupto da população

A dimensão da população/gerações também deve ter a ver com o nº de soluções do problema

Nota adicional: a diversidade da população é um aspeto importante (por exemplo evitar ter sempre o mesmo valor no gene de uma dada posição, como vimos nos exemplos)

Algoritmos Genéticos – Parametrização
Taxa de Cruzamento

A Taxa de Cruzamento vai definir a probabilidade com a qual um individuo ou um par de indivíduos serão cruzados para gerar nova descendência

Uma Taxa de Cruzamento baixa faz com que as gerações da população sejam pouco renovadas, fazendo com que a evolução seja muito lenta

Uma Taxa de Cruzamento elevada origina uma grande renovação de geração para geração, o que se por um lado pode ser positivo para evitar o problema descrito anteriormente, pode também originar a necessidade de substituição de indivíduos com aptidão elevada

Um valor típico para a taxa de cruzamento situa-se entre os 60% e 80%

Algoritmos Genéticos – Parametrização
Taxa de Mutação

A mutação é uma operação importante pelo facto de poder gerar indivíduos com melhores aptidões, fugindo desse modo de máximos locais do problema. Dito de outro modo, com a mutação podemos introduzir um fator adicional de diversidade na população

Uma taxa de mutação muito elevada conduz a uma pesquisa aleatória que poderá não originar convergência, visto que o mais provável numa mutação é até gerar indivíduos com pior aptidão

Os valores típicos da taxa de mutação são bastante reduzidos (por exemplo, cerca de 1%)

Algoritmos Genéticos – Parametrização
Taxa de Substituição

A Taxa de Substituição define a percentagem de população que será substituída em cada transição de gerações

Um valor muito elevado leva a que possa ocorrer uma renovação excessiva da população enquanto que um valor muito baixo origina uma estagnação da população

A substituição não deve ser puramente elitista (evitar máximos locais) mas também não deve ignorar passar alguns do melhores indivíduos da população para a geração seguinte

A taxa de substituição deve estar de algum modo relacionada com a taxa de cruzamento, não adianta fazer muitos cruzamentos e ter uma taxa de substituição baixa nem fazer poucos cruzamentos e ter uma taxa de substituição elevada

Algoritmos Genéticos – Parametrização
Critérios de Paragem

Geralmente adota-se um dos seguintes critérios de paragem:

- quando se atinge um determinado número de gerações
- quando se atinge uma solução com um determinado valor de aptidão (para o tipo de problemas em que se pretende encontrar uma solução com custo inferior a um dado valor mas sem necessidade de encontrar a melhor solução)
- quando se atinge um determinado limite temporal (para problemas que exigem soluções em tempo útil ou em tempo real)
- quando a população estabiliza num dado nº seguido de gerações

Algoritmos Genéticos – Adaptação e Avaliação

As noções de avaliação e de aptidão são por vezes intercambiáveis. No entanto, deve-se distinguir a noção de função de avaliação e função de aptidão, usadas nos Algoritmos Genéticos

A Avaliação proporciona uma medida de desempenho em relação a um conjunto particular de parâmetros, sendo que, a avaliação de um dado indivíduo é independente dos outros indivíduos.

A Aptidão transforma essa medida de desempenho na atribuição de oportunidades de sobrevivência e reprodução, sendo, portanto, definida em relação aos outros indivíduos da população

O cálculo das funções de avaliação e de aptidão não deve ser muito complexo e demorado

Algoritmos Genéticos – População Inicial

A população inicial pode ser gerada aleatoriamente

Devem ser evitados indivíduos repetidos

Faz sentido “olhar” com algum sentido crítico para a diversidade genética dos elementos gerados na população, por exemplo no caso do 2º exemplo a tarefa t2 aparecia sempre na posição nº 3 o que deveria ser evitado

É também adequado criar alguns indivíduos da população inicial através de heurísticas, por exemplo, no caso das 5 tarefas poderíamos ter usado o EDD (Early Due Date) e/ou o SPT (Shortest Processing Time) para gerar alguns elementos para a população inicial

Algoritmos Genéticos – Seleção

A escolha dos indivíduos melhor avaliados da população através de um operador de seleção determinístico pode parecer lógica

Tal escolha, dita elitista, pode, no entanto, levar à redução da diversidade da população, fazendo com que uma boa solução obtida no principio se torne dominante e evitando a exploração adequada do espaço de pesquisa

É importante que os indivíduos pior avaliados e menos adaptados tenham probabilidade, mesmo que reduzida, de serem selecionados

Algoritmos Genéticos – Seleção

Um dos métodos de seleção proporcional mais comuns é o método da roleta (“Roulette Wheel Parent Selection”)segundo o qual a cada indivíduo i da população é atribuída uma probabilidade de seleção Ps(i), calculada através do quociente entre a sua avaliação e o somatório das avaliações de todos os indivíduos da população, ou por uma fórmula similar (por vezes subtrai-se ao numerador e denominador o mínimo valor obtido para a avaliação de um indivíduo da população)

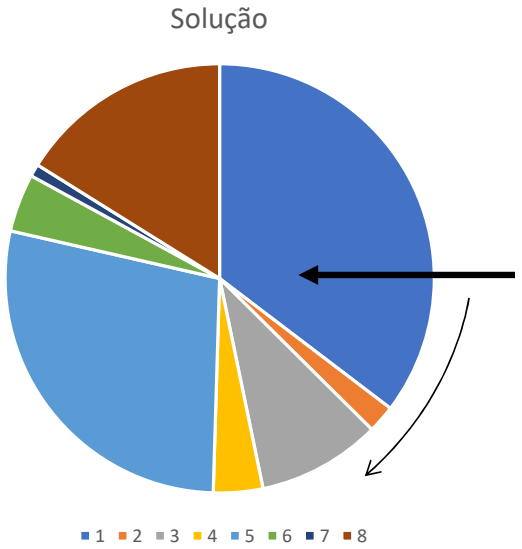
Segundo esse método, a seleção de indivíduos é realizada através da simulação de n rotações de uma roleta cujas fatias tivessem um ângulo proporcional ao resultado da fórmula referida anteriormente, constituindo assim a população pretendida. Ou seja, os indivíduos com melhor de avaliação terão mais possibilidades de serem escolhidos, no entanto, nada garante que o sejam, nem nada implica que o elemento com pior valor de avaliação não possa ser escolhido

Algoritmos Genéticos – Seleção

Elemento nº	População após cruzamento	Solução x	f(x)
1	100101	37	1369
2	001001	9	81
3	010011	19	361
4	001100	12	144
5 - cruz.1,2	100001	33	1089
6 - cruz.1,2	001101	13	169
7 - cruz.3,4	000110	6	36
8 - cruz.3,4	011001	25	625



Roulette Wheel Parent Selection



Algoritmos Genéticos – Seleção

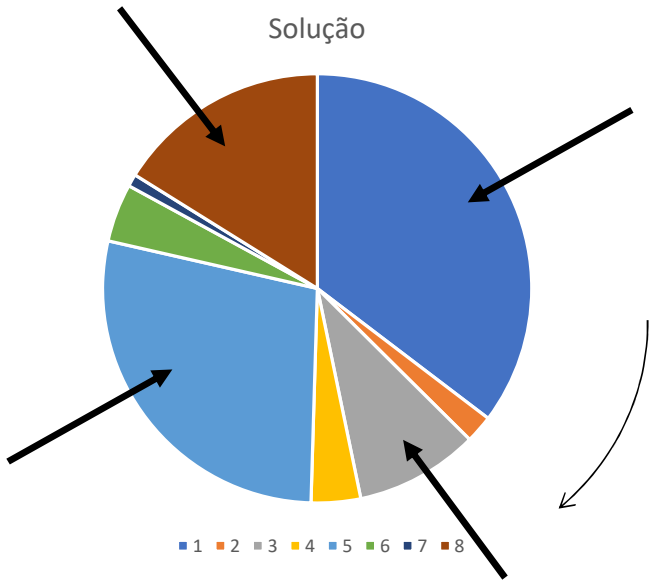
Além do método da roleta foram ainda estudados outros operadores de seleção proporcional, tais como, por exemplo, o Método de Seleção Universal Estocástica

Rodamos a roleta apenas uma vez e temos um conjunto de “ponteiros” igualmente espaçados, os indivíduos cujas fatias estejam localizadas em tais ponteiros serão escolhidos

Algoritmos Genéticos – Seleção

Elemento nº	População após cruzamento	Solução x	f(x)
1	100101	37	1369
2	001001	9	81
3	010011	19	361
4	001100	12	144
5 - cruz.1,2	100001	33	1089
6 - cruz.1,2	001101	13	169
7 - cruz.3,4	000110	6	36
8 - cruz.3,4	011001	25	625

Método de Seleção Universal Estocástica



Algoritmos Genéticos – Seleção

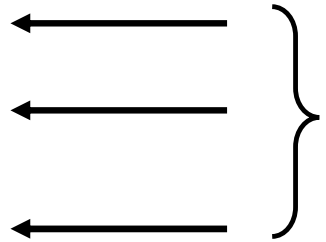
O Método de seleção por Torneios (“Tournament Selection”) é um método de seleção por ordem de classificação que consiste na organização de n torneios entre 2 ou mais indivíduos da população, escolhidos aleatoriamente

O indivíduo selecionado é o que em termos de avaliação estiver em posição de superioridade no confronto direto entre os valores de avaliação

Algoritmos Genéticos – Seleção

Tournament Selection

Elemento nº	População após cruzamento	Solução x	f(x)
1	100101	37	1369
2	001001	9	81
3	010011	19	361
4	001100	12	144
5 - cruz.1,2	100001	33	1089
6 - cruz.1,2	001101	13	169
7 - cruz.3,4	000110	6	36
8 - cruz.3,4	011001	25	625



passa o elemento 5
(neste caso é visada uma maximização)